

CLAUDE CERTIFIED ARCHITECT

Question Bank — 610 Original Practice Questions with Answers

An unofficial, exhaustive practice-question bank for the CCA-Foundations exam: roughly 610 original, scenario-based multiple-choice questions, split across all five blueprint domains, each with the correct answer marked and a one-line rationale.

Domain	Weight	Questions
1. Agent Architecture & Orchestration	27%	138
2. Claude Code Configuration & Workflows	20%	120
3. Prompt Engineering & Structured Output	20%	118
4. Tool Design & MCP Integration	18%	118
5. Context Management & Reliability	15%	116
Total		610

How to use this bank: Work a domain in one sitting, cover the rationale column, and check yourself against it after answering. Questions within a domain intentionally reuse a smaller set of underlying concepts across many concrete scenarios — that repetition is deliberate. If a pattern (e.g., “fix the description before adding a classifier”) still trips you up on its fifth appearance, that’s the signal to go back to the concept guide, not to memorize the specific scenario.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish. It is designed to test the underlying reasoning the exam is publicly reported to probe.

Contents

Domain 1 — Agent Architecture & Orchestration	138 Qs
Domain 2 — Claude Code Configuration & Workflows	120 Qs
Domain 3 — Prompt Engineering & Structured Output	118 Qs
Domain 4 — Tool Design & MCP Integration	118 Qs
Domain 5 — Context Management & Reliability	116 Qs

Agent Architecture & Orchestration

Q1. A model response returns stop_reason: "tool_use". What should the orchestration loop do?

- A. Stop the loop immediately.
- B. Treat the response as truncated and resend the identical prompt.
- C. Discard the response and ask the user to rephrase.
- D. ✓ Execute the requested tool call(s), return tool_result blocks, and continue the loop.

Correct answer: D. tool_use means the model wants a tool executed; run it, return the result, and keep looping.

Q2. A model response returns stop_reason: "end_turn". What should the orchestration loop do?

- A. Automatically retry with a longer prompt.
- B. Wait for a second stop_reason before stopping.
- C. ✓ Stop the loop — the model has produced its final answer.
- D. Execute any tool-shaped text found in the response.

Correct answer: C. end_turn signals genuine completion; continuing the loop afterward wastes calls and can cause drift.

Q3. A model response returns stop_reason: "max_tokens". What should the orchestration loop do?

- A. Restart the entire task from scratch automatically.
- B. ✓ Treat the response as truncated, not complete, and avoid acting on it as a final answer.
- C. Treat it exactly like end_turn.
- D. Silently discard the partial output with no follow-up.

Correct answer: B. max_tokens means generation was cut off; treating a truncated response as complete risks acting on partial output.

Q4. A model response returns stop_reason: "stop_sequence". What should the orchestration loop do?

- A. ✓ Recognize a configured stop string was hit and handle it per its intended purpose.
- B. Treat it as an unrecoverable error requiring escalation.
- C. Ignore it and keep generating anyway.
- D. Treat it identically to max_tokens.

Correct answer: A. stop_sequence is a deliberate, configured condition — not an error and not the same as truncation.

Q5. A coordinator is asked to research "AI's impact on creative industries" and decomposes it into subagent assignments covering only three visual-art subtopics (digital art, graphic design, photography). Each subagent does accurate work; the final report reads as complete but never touches music, writing, film, and performance. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "AI's impact on creative industries" to three visual-art subtopics (digital art, graphic design, photography) at decomposition time and never assigned coverage of music, writing, film, and performance. Root-cause the decomposition step first.

Q6. A coordinator is asked to research "global supply chain risk" and decomposes it into subagent assignments covering only shipping and logistics delays. Each subagent does accurate work; the final report reads as complete but never touches supplier concentration, geopolitical, and currency risk. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "global supply chain risk" to shipping and logistics delays at decomposition time and never assigned coverage of supplier concentration, geopolitical, and currency risk. Root-cause the decomposition step first.

Q7. A coordinator is asked to research "quarterly customer sentiment" and decomposes it into subagent assignments covering only NPS survey scores. Each subagent does accurate work; the final report reads as complete but never touches support-ticket sentiment and social-media mentions. Where is the fault?

- A. The user, for an under-specified original prompt.
- B. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- C. The subagents, for not asking clarifying questions.
- D. The synthesis step, for not flagging the gap.

Correct answer: B. The subagents executed exactly what they were assigned; the coordinator narrowed "quarterly customer sentiment" to NPS survey scores at decomposition time and never assigned coverage of support-ticket sentiment and social-media mentions. Root-cause the decomposition step first.

Q8. A coordinator is asked to research "competitive landscape analysis" and decomposes it into subagent assignments covering only pricing comparisons. Each subagent does accurate work; the final report reads as complete but never touches feature gaps, market share, and hiring trends. Where is the fault?

- A. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- B. The subagents, for not asking clarifying questions.
- C. The synthesis step, for not flagging the gap.
- D. The user, for an under-specified original prompt.

Correct answer: A. The subagents executed exactly what they were assigned; the coordinator narrowed "competitive landscape analysis" to pricing comparisons at decomposition time and never assigned coverage of feature gaps, market share, and hiring trends. Root-cause the decomposition step first.

Q9. A coordinator is asked to research "organizational security posture" and decomposes it into subagent assignments covering only network security. Each subagent does accurate work; the final report reads as complete but never touches endpoint, identity, and physical security. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "organizational security posture" to network security at decomposition time and never assigned coverage of endpoint, identity, and physical security. Root-cause the decomposition step first.

Q10. A coordinator is asked to research "employee engagement" and decomposes it into subagent assignments covering only exit-interview themes. Each subagent does accurate work; the final report reads as complete but never touches pulse-survey results and manager 1:1 notes. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "employee engagement" to exit-interview themes at decomposition time and never assigned coverage of pulse-survey results and manager 1:1 notes. Root-cause the decomposition step first.

Q11. A coordinator is asked to research "market entry strategy" and decomposes it into subagent assignments covering only regulatory requirements. Each subagent does accurate work; the final report reads as complete but never touches competitive intensity and customer demand signals. Where is the fault?

- A. The user, for an under-specified original prompt.
- B. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- C. The subagents, for not asking clarifying questions.
- D. The synthesis step, for not flagging the gap.

Correct answer: B. The subagents executed exactly what they were assigned; the coordinator narrowed "market entry strategy" to regulatory requirements at decomposition time and never assigned coverage of competitive intensity and customer demand signals. Root-cause the decomposition step first.

Q12. A coordinator is asked to research "product roadmap review" and decomposes it into subagent assignments covering only engineering feasibility. Each subagent does accurate work; the final report reads as complete but never touches customer demand and go-to-market readiness. Where is the fault?

- A. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- B. The subagents, for not asking clarifying questions.
- C. The synthesis step, for not flagging the gap.
- D. The user, for an under-specified original prompt.

Correct answer: A. The subagents executed exactly what they were assigned; the coordinator narrowed "product roadmap review" to engineering feasibility at decomposition time and never assigned coverage of customer demand and go-to-market readiness. Root-cause the decomposition step first.

Q13. A coordinator is asked to research "brand health" and decomposes it into subagent assignments covering only social-media sentiment. Each subagent does accurate work; the final report reads as complete but never touches press coverage, search trends, and customer surveys. Where is the fault?

- A. The subagents, for not asking clarifying questions.
- B. The synthesis step, for not flagging the gap.
- C. The user, for an under-specified original prompt.
- D. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.

Correct answer: D. The subagents executed exactly what they were assigned; the coordinator narrowed "brand health" to social-media sentiment at decomposition time and never assigned coverage of press coverage, search trends, and customer surveys. Root-cause the decomposition step first.

Q14. A coordinator is asked to research "vendor risk assessment" and decomposes it into subagent assignments covering only financial stability. Each subagent does accurate work; the final report reads as complete but never touches data-security posture and delivery-performance history. Where is the fault?

- A. The synthesis step, for not flagging the gap.
- B. The user, for an under-specified original prompt.
- C. ✓ The coordinator, which silently narrowed a broad topic to a convenient subset at decomposition time.
- D. The subagents, for not asking clarifying questions.

Correct answer: C. The subagents executed exactly what they were assigned; the coordinator narrowed "vendor risk assessment" to financial stability at decomposition time and never assigned coverage of data-security posture and delivery-performance history. Root-cause the decomposition step first.

Q15. A production system must guarantee that refunds above a fixed dollar threshold require human approval. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q16. A production system must guarantee that personally identifiable information must be redacted before any logging call. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q17. A production system must guarantee that a production deploy requires two independent approvals. Which enforcement mechanism is most reliable?

- A. A firmly worded instruction in the system prompt.
- B. A note in CLAUDE.md asking the model to remember the rule.
- C. Asking the model to double-check itself before calling the tool.
- D. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.

Correct answer: D. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q18. A production system must guarantee that expense reports over \$1,000 require manager sign-off. Which enforcement mechanism is most reliable?

- A. A note in CLAUDE.md asking the model to remember the rule.
- B. Asking the model to double-check itself before calling the tool.
- C. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- D. A firmly worded instruction in the system prompt.

Correct answer: C. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q19. A production system must guarantee that a prescribed medication dosage must never exceed the labeled maximum. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q20. A production system must guarantee that wire transfers above \$10,000 require secondary verification. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q21. A production system must guarantee that discount codes may never exceed a 30% markdown. Which enforcement mechanism is most reliable?

- A. A firmly worded instruction in the system prompt.
- B. A note in CLAUDE.md asking the model to remember the rule.
- C. Asking the model to double-check itself before calling the tool.
- D. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.

Correct answer: D. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q22. A production system must guarantee that account deletion requires an explicit, separate confirmation step. Which enforcement mechanism is most reliable?

- A. A note in CLAUDE.md asking the model to remember the rule.
- B. Asking the model to double-check itself before calling the tool.
- C. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- D. A firmly worded instruction in the system prompt.

Correct answer: C. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q23. A production system must guarantee that customer data may never be exported outside its home region. Which enforcement mechanism is most reliable?

- A. Asking the model to double-check itself before calling the tool.
- B. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- C. A firmly worded instruction in the system prompt.
- D. A note in CLAUDE.md asking the model to remember the rule.

Correct answer: B. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q24. A production system must guarantee that API credentials must be rotated before being granted to a new service. Which enforcement mechanism is most reliable?

- A. ✓ A PreToolUse hook that programmatically blocks the disqualifying tool call before it executes.
- B. A firmly worded instruction in the system prompt.
- C. A note in CLAUDE.md asking the model to remember the rule.
- D. Asking the model to double-check itself before calling the tool.

Correct answer: A. Programmatic enforcement (a hook) fires deterministically on every call regardless of how the model was prompted; prompt wording alone is probabilistic and can be talked around at the edges.

Q25. Three subagents each research an independent, unrelated topic with no shared dependency. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. Pipeline with no ordering constraints.
- D. ✓ Parallel — the subtasks are independent.

Correct answer: D. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q26. A validation subagent must confirm data quality before a writing subagent drafts the report. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. ✓ Sequential — writing depends on validation's result.
- D. Parallel — the subtasks are independent.

Correct answer: C. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q27. One subagent fetches a stock price; a second subagent uses that price to compute portfolio value. How should these subtasks be executed?

- A. Peer-to-peer — the subagents should negotiate directly.
- B. ✓ Sequential — the second step needs the first step's output.
- C. Parallel — the subtasks are independent.
- D. Sequential — a later step depends on an earlier step's result.

Correct answer: B. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q28. Five independent log files are each scanned for anomalies by separate subagents. How should these subtasks be executed?

- A. ✓ Parallel — no subagent depends on another's output.
- B. Parallel — the subtasks are independent.
- C. Sequential — a later step depends on an earlier step's result.
- D. Peer-to-peer — the subagents should negotiate directly.

Correct answer: A. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q29. A draft is produced by one subagent, then reviewed by a second, independent subagent. How should these subtasks be executed?

- A. Parallel — the subtasks are independent.
- B. Sequential — a later step depends on an earlier step's result.
- C. Peer-to-peer — the subagents should negotiate directly.
- D. ✓ Sequential — review requires the draft to exist first.

Correct answer: D. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q30. Three subagents each call a different city's weather API with no shared state. How should these subtasks be executed?

- A. Sequential — a later step depends on an earlier step's result.
- B. Peer-to-peer — the subagents should negotiate directly.
- C. ✓ Parallel — fully independent calls.
- D. Parallel — the subtasks are independent.

Correct answer: C. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q31. Data must be extracted, then transformed, then loaded into a warehouse. How should these subtasks be executed?

- A. Peer-to-peer — the subagents should negotiate directly.
- B. ✓ Sequential — a classic pipeline with strict ordering.
- C. Parallel — the subtasks are independent.
- D. Sequential — a later step depends on an earlier step's result.

Correct answer: B. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q32. Four subagents each summarize a different, unrelated news article. How should these subtasks be executed?

- A. ✓ Parallel — the summaries don't depend on each other.
- B. Parallel — the subtasks are independent.
- C. Sequential — a later step depends on an earlier step's result.
- D. Peer-to-peer — the subagents should negotiate directly.

Correct answer: A. Independent subtasks with no shared dependency are strong candidates for parallel execution; a subtask that genuinely needs a prior step's output must run sequentially after it.

Q33. Mid-execution, a subagent returns an unexpected null result partway through the task. What should the coordinator do?

- A. Continue executing the original fixed plan unchanged.
- B. Abort the entire task immediately with no partial output.
- C. Ignore the new information and let a downstream step handle it silently.
- D. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.

Correct answer: D. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q34. Mid-execution, new information emerges mid-research that changes the relevant scope. What should the coordinator do?

- A. Abort the entire task immediately with no partial output.
- B. Ignore the new information and let a downstream step handle it silently.
- C. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- D. Continue executing the original fixed plan unchanged.

Correct answer: C. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q35. Mid-execution, a tool call reveals the task is substantially larger than originally estimated. What should the coordinator do?

- A. Ignore the new information and let a downstream step handle it silently.
- B. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- C. Continue executing the original fixed plan unchanged.
- D. Abort the entire task immediately with no partial output.

Correct answer: B. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q36. Mid-execution, the user changes a requirement while the task is already in progress. What should the coordinator do?

- A. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- B. Continue executing the original fixed plan unchanged.
- C. Abort the entire task immediately with no partial output.
- D. Ignore the new information and let a downstream step handle it silently.

Correct answer: A. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q37. Mid-execution, a subtask has failed twice after reasonable retries. What should the coordinator do?

- A. Continue executing the original fixed plan unchanged.
- B. Abort the entire task immediately with no partial output.
- C. Ignore the new information and let a downstream step handle it silently.
- D. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.

Correct answer: D. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q38. Mid-execution, a data source the plan depended on turns out to be deprecated. What should the coordinator do?

- A. Abort the entire task immediately with no partial output.
- B. Ignore the new information and let a downstream step handle it silently.
- C. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- D. Continue executing the original fixed plan unchanged.

Correct answer: C. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q39. Mid-execution, the task is approaching its allocated budget or token ceiling. What should the coordinator do?

- A. Ignore the new information and let a downstream step handle it silently.
- B. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- C. Continue executing the original fixed plan unchanged.
- D. Abort the entire task immediately with no partial output.

Correct answer: B. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q40. Mid-execution, two subagents return results that directly conflict with each other. What should the coordinator do?

- A. ✓ Revise the plan based on the new information — dynamic replanning, not blind continuation.
- B. Continue executing the original fixed plan unchanged.
- C. Abort the entire task immediately with no partial output.
- D. Ignore the new information and let a downstream step handle it silently.

Correct answer: A. Good orchestration treats the plan as a living object: when intermediate results change what's known, the coordinator should re-evaluate and adjust rather than executing blindly to a stale plan.

Q41. A fixed sequence of steps — parse input, call one classification model, format output — with no runtime branching. How should this system be classified?

- A. Agent
- B. Conversational system
- C. Single-shot prompt with no loop
- D. ✓ Workflow

Correct answer: D. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q42. A system where the model decides which tools to call, in what order, and when the task is done, adapting as it goes. How should this system be classified?

- A. Conversational system
- B. Single-shot prompt with no loop
- C. ✓ Agent
- D. Workflow

Correct answer: C. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q43. A chat interface where a human drives every turn and the model never acts without being prompted each time. How should this system be classified?

- A. Single-shot prompt with no loop
- B. ✓ Conversational system
- C. Agent
- D. Workflow

Correct answer: B. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q44. A nightly ETL job that always runs the same three transformation steps regardless of the data. How should this system be classified?

- A. ✓ Workflow
- B. Agent
- C. Conversational system
- D. Single-shot prompt with no loop

Correct answer: A. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q45. A research assistant that decides on its own how many sources to check and when it has enough to answer. How should this system be classified?

- A. Workflow
- B. Conversational system
- C. Single-shot prompt with no loop
- D. ✓ Agent

Correct answer: D. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q46. A support bot that only ever responds to what the user just typed, never initiating its own actions. How should this system be classified?

- A. Workflow
- B. Single-shot prompt with no loop
- C. ✓ Conversational system
- D. Agent

Correct answer: C. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q47. A system that dynamically decides to spin up additional subagents when a subtask proves more complex than expected. How should this system be classified?

- A. Single-shot prompt with no loop
- B. ✓ Agent
- C. Workflow
- D. Conversational system

Correct answer: B. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q48. A fixed onboarding checklist bot that always asks the same five questions in the same order. How should this system be classified?

- A. ✓ Workflow
- B. Agent
- C. Conversational system
- D. Single-shot prompt with no loop

Correct answer: A. Classification hinges on runtime autonomy: fixed steps with no model-driven branching is a workflow; human-driven turn-by-turn exchange is conversational; model-directed tool use and adaptive planning is agentic.

Q49. A fully-specified ETL pipeline with one LLM classification step in the middle. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes — flip the framing regardless of the scenario's actual structure.
- B. Yes, agentic architecture is always the safer default choice.
- C. No, because LLMs cannot be trusted with any autonomy.
- D. ✓ No — the steps are fully knowable ahead of time.

Correct answer: D. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q50. An open-ended competitive research task where the right sources aren't known in advance. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes, agentic architecture is always the safer default choice.
- B. No, because LLMs cannot be trusted with any autonomy.
- C. ✓ Yes — scope is open-ended and requires adaptive planning.
- D. No — flip the framing regardless of the scenario's actual structure.

Correct answer: C. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q51. A fixed weekly report that always pulls the same three metrics in the same format. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. No, because LLMs cannot be trusted with any autonomy.
- B. ✓ No — the sequence never varies and needs no runtime judgment.
- C. Yes — flip the framing regardless of the scenario's actual structure.
- D. Yes, agentic architecture is always the safer default choice.

Correct answer: B. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q52. A customer-support task that may require zero, one, or several tool calls depending on the issue. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. ✓ Yes — the number and order of tool calls can't be fixed in advance.
- B. No — flip the framing regardless of the scenario's actual structure.
- C. Yes, agentic architecture is always the safer default choice.
- D. No, because LLMs cannot be trusted with any autonomy.

Correct answer: A. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q53. A single well-structured prompt that reliably classifies a support ticket into one of five categories. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes — flip the framing regardless of the scenario's actual structure.
- B. Yes, agentic architecture is always the safer default choice.
- C. No, because LLMs cannot be trusted with any autonomy.
- D. ✓ No — a simple prompt already solves this without added orchestration.

Correct answer: D. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q54. A multi-step investigation where each finding determines what to look into next. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. Yes, agentic architecture is always the safer default choice.
- B. No, because LLMs cannot be trusted with any autonomy.
- C. ✓ Yes — later steps genuinely depend on adapting to earlier findings.
- D. No — flip the framing regardless of the scenario's actual structure.

Correct answer: C. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q55. A nightly batch job with a rigid, unchanging list of files to process in order. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. No, because LLMs cannot be trusted with any autonomy.
- B. ✓ No — there is no need for model-driven autonomy over the sequence.
- C. Yes — flip the framing regardless of the scenario's actual structure.
- D. Yes, agentic architecture is always the safer default choice.

Correct answer: B. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q56. A research task where the number of necessary subagents can't be known until initial findings come in. Is agentic architecture (versus a simpler fixed workflow or single prompt) justified here?

- A. ✓ Yes — the plan must adapt to information only available mid-task.
- B. No — flip the framing regardless of the scenario's actual structure.
- C. Yes, agentic architecture is always the safer default choice.
- D. No, because LLMs cannot be trusted with any autonomy.

Correct answer: A. Agentic architecture earns its added latency, cost, and failure surface when scope is open-ended or the plan must adapt at runtime; fully-knowable, fixed sequences are better served by a simpler workflow.

Q57. In a hub-and-spoke multi-agent system, who owns: "Task decomposition and scoping"?

- A. Subagent
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Coordinator

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q58. In a hub-and-spoke multi-agent system, who owns: "Packing relevant context into each subagent's task prompt"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q59. In a hub-and-spoke multi-agent system, who owns: "Executing one narrow, well-scoped subtask"?

- A. Neither — this belongs to a human
- B. ✓ Subagent
- C. Coordinator
- D. Both equally

Correct answer: B. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q60. In a hub-and-spoke multi-agent system, who owns: "Cross-subagent error handling and retry policy"?

- A. ✓ Coordinator
- B. Subagent
- C. Both equally
- D. Neither — this belongs to a human

Correct answer: A. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q61. In a hub-and-spoke multi-agent system, who owns: "Final result aggregation and synthesis"?

- A. Subagent
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Coordinator

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q62. In a hub-and-spoke multi-agent system, who owns: "Deciding whether to escalate to a human"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q63. In a hub-and-spoke multi-agent system, who owns: "Delegating a subtask to the right subagent"?

- A. Neither — this belongs to a human
- B. ✓ Coordinator
- C. Subagent
- D. Both equally

Correct answer: B. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q64. In a hub-and-spoke multi-agent system, who owns: "Defining the exact scope boundaries of a subtask"?

- A. ✓ Coordinator
- B. Subagent
- C. Both equally
- D. Neither — this belongs to a human

Correct answer: A. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q65. In a hub-and-spoke multi-agent system, who owns: "Reporting its own result back without inventing scope beyond its assignment"?

- A. Coordinator
- B. Both equally
- C. Neither — this belongs to a human
- D. ✓ Subagent

Correct answer: D. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q66. In a hub-and-spoke multi-agent system, who owns: "Reconciling conflicting results returned by two subagents"?

- A. Both equally
- B. Neither — this belongs to a human
- C. ✓ Coordinator
- D. Subagent

Correct answer: C. The coordinator owns decomposition, delegation, error handling, escalation, and synthesis; subagents own only the narrow execution of what they were explicitly assigned.

Q67. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. demonstrated inability to make progress after reasonable attempts
- B. ✓ a sentiment classifier flagging the conversation as negative
- C. an explicit user request for a human
- D. a genuine policy gap the agent has no authority to interpret

Correct answer: B. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q68. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. ✓ the model's self-reported confidence score
- B. an explicit user request for a human
- C. a genuine policy gap the agent has no authority to interpret
- D. an automatic soft-label classifier's complexity score

Correct answer: A. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q69. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. a genuine policy gap the agent has no authority to interpret
- B. demonstrated inability to make progress after reasonable attempts
- C. the model's self-reported confidence score
- D. ✓ a sentiment classifier flagging the conversation as negative

Correct answer: D. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q70. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. the model's self-reported confidence score
- B. an automatic soft-label classifier's complexity score
- C. ✓ a sentiment classifier flagging the conversation as negative
- D. an explicit user request for a human

Correct answer: C. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q71. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. a sentiment classifier flagging the conversation as negative
- B. ✓ an automatic soft-label classifier's complexity score
- C. demonstrated inability to make progress after reasonable attempts
- D. an explicit user request for a human

Correct answer: B. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q72. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. ✓ the model's self-reported confidence score
- B. a genuine policy gap the agent has no authority to interpret
- C. an explicit user request for a human
- D. an automatic soft-label classifier's complexity score

Correct answer: A. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q73. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. demonstrated inability to make progress after reasonable attempts
- B. the model's self-reported confidence score
- C. an explicit user request for a human
- D. ✓ a sentiment classifier flagging the conversation as negative

Correct answer: D. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q74. Which of the following is the LEAST reliable trigger for escalating an agent interaction to a human?

- A. an automatic soft-label classifier's complexity score
- B. a genuine policy gap the agent has no authority to interpret
- C. ✓ a sentiment classifier flagging the conversation as negative
- D. the model's self-reported confidence score

Correct answer: C. Sentiment analysis, self-reported model confidence, and soft classifier scores are all internally-generated or statistical signals that can be miscalibrated; explicit requests and structurally verifiable conditions are the reliable category.

Q75. A subagent produces a plan that exceeds the client's stated budget, even though the coordinator seemingly "knew" that constraint. Why?

- A. The synthesis step corrupted a correct subagent result.
- B. ✓ The coordinator never explicitly passed a budget constraint into the subagent's task prompt.
- C. The subagent ignored instructions it was actually given.
- D. The model's context window was too small to hold the task.

Correct answer: B. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q76. A subagent duplicates research another subagent already completed, even though the coordinator seemingly "knew" that constraint. Why?

- A. ✓ The coordinator never explicitly passed prior findings from an earlier subagent into the subagent's task prompt.
- B. The subagent ignored instructions it was actually given.
- C. The model's context window was too small to hold the task.
- D. The synthesis step corrupted a correct subagent result.

Correct answer: A. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q77. A subagent returns free text instead of the structured format the coordinator needed, even though the coordinator seemingly "knew" that constraint. Why?

- A. The subagent ignored instructions it was actually given.
- B. The model's context window was too small to hold the task.
- C. The synthesis step corrupted a correct subagent result.
- D. ✓ The coordinator never explicitly passed the required output format into the subagent's task prompt.

Correct answer: D. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q78. A subagent produces a timeline that ignores the actual due date, even though the coordinator seemingly "knew" that constraint. Why?

- A. The model's context window was too small to hold the task.
- B. The synthesis step corrupted a correct subagent result.
- C. ✓ The coordinator never explicitly passed a hard deadline into the subagent's task prompt.
- D. The subagent ignored instructions it was actually given.

Correct answer: C. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q79. A subagent writes in a tone that doesn't match who the report is for, even though the coordinator seemingly "knew" that constraint. Why?

- A. The synthesis step corrupted a correct subagent result.
- B. ✓ The coordinator never explicitly passed the target audience into the subagent's task prompt.
- C. The subagent ignored instructions it was actually given.
- D. The model's context window was too small to hold the task.

Correct answer: B. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q80. A subagent repeats the exact approach that already failed once, even though the coordinator seemingly "knew" that constraint. Why?

- A. ✓ The coordinator never explicitly passed context about a prior failed attempt into the subagent's task prompt.
- B. The subagent ignored instructions it was actually given.
- C. The model's context window was too small to hold the task.
- D. The synthesis step corrupted a correct subagent result.

Correct answer: A. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q81. A subagent answers a narrower question than what was actually asked, even though the coordinator seemingly "knew" that constraint. Why?

- A. The subagent ignored instructions it was actually given.
- B. The model's context window was too small to hold the task.
- C. The synthesis step corrupted a correct subagent result.
- D. ✓ The coordinator never explicitly passed the user's original underlying question into the subagent's task prompt.

Correct answer: D. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q82. A subagent produces output that violates house style rules, even though the coordinator seemingly "knew" that constraint. Why?

- A. The model's context window was too small to hold the task.
- B. The synthesis step corrupted a correct subagent result.
- C. ✓ The coordinator never explicitly passed the required style guide into the subagent's task prompt.
- D. The subagent ignored instructions it was actually given.

Correct answer: C. Subagents run in isolated context and inherit nothing automatically from the coordinator; any constraint the coordinator doesn't explicitly restate in the task prompt effectively does not exist for that subagent.

Q83. A central agent needs to decompose one broad task into independent pieces and merge the results. Which multi-agent topology best fits?

- A. Fully decentralized swarm with no defined roles
- B. ✓ Hub-and-spoke
- C. Pipeline
- D. Peer-to-peer

Correct answer: B. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q84. Data must flow through extract, validate, and format stages in strict, unchanging order. Which multi-agent topology best fits?

- A. ✓ Pipeline
- B. Hub-and-spoke
- C. Peer-to-peer
- D. Fully decentralized swarm with no defined roles

Correct answer: A. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q85. Several agents need to negotiate directly with each other with no single owner of the outcome. Which multi-agent topology best fits?

- A. Hub-and-spoke
- B. Pipeline
- C. Fully decentralized swarm with no defined roles
- D. ✓ Peer-to-peer

Correct answer: D. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q86. A researcher agent, a fact-checker agent, and a writer agent must each build on the previous stage's output. Which multi-agent topology best fits?

- A. Peer-to-peer
- B. Fully decentralized swarm with no defined roles
- C. ✓ Pipeline
- D. Hub-and-spoke

Correct answer: C. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q87. A single coordinator manages five independent research streams and combines them into one report. Which multi-agent topology best fits?

- A. Fully decentralized swarm with no defined roles
- B. ✓ Hub-and-spoke
- C. Pipeline
- D. Peer-to-peer

Correct answer: B. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q88. Two specialized agents must jointly negotiate a shared plan with no coordinator arbitrating. Which multi-agent topology best fits?

- A. ✓ Peer-to-peer
- B. Hub-and-spoke
- C. Pipeline
- D. Fully decentralized swarm with no defined roles

Correct answer: A. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q89. A support system routes each ticket to exactly one of several independent specialist subagents. Which multi-agent topology best fits?

- A. Pipeline
- B. Peer-to-peer
- C. Fully decentralized swarm with no defined roles
- D. ✓ Hub-and-spoke

Correct answer: D. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q90. A document must pass sequentially through drafting, legal review, and final formatting. Which multi-agent topology best fits?

- A. Peer-to-peer
- B. Fully decentralized swarm with no defined roles
- C. ✓ Pipeline
- D. Hub-and-spoke

Correct answer: C. Hub-and-spoke fits central decomposition/aggregation, pipeline fits strict sequential handoff, and peer-to-peer fits decentralized negotiation with no single owning coordinator.

Q91. A coordinator caps subagent loops at 10 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Increase the cap indefinitely until the loop naturally stops.
- B. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- C. Treat reaching the cap as successful task completion.
- D. Silently return whatever partial output exists with no flag.

Correct answer: B. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q92. A coordinator caps subagent loops at 15 iterations as a safety ceiling. What should happen when that cap is hit?

- A. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- B. Treat reaching the cap as successful task completion.
- C. Silently return whatever partial output exists with no flag.
- D. Increase the cap indefinitely until the loop naturally stops.

Correct answer: A. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q93. A coordinator caps subagent loops at 20 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Treat reaching the cap as successful task completion.
- B. Silently return whatever partial output exists with no flag.
- C. Increase the cap indefinitely until the loop naturally stops.
- D. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.

Correct answer: D. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q94. A coordinator caps subagent loops at 25 iterations as a safety ceiling. What should happen when that cap is hit?

- A. Silently return whatever partial output exists with no flag.
- B. Increase the cap indefinitely until the loop naturally stops.
- C. ✓ Treat the hit ceiling as a distinct failure mode to surface, not a silent success.
- D. Treat reaching the cap as successful task completion.

Correct answer: C. A hit iteration cap is a safety ceiling, not a completion signal — it should trigger explicit handling, not be silently treated as if the task finished normally.

Q95. A ticket classification task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Switch to a larger model to reduce the number of steps needed.
- B. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- C. Add more subagents to parallelize the cost away.
- D. Ignore cost — agentic systems should always maximize autonomy.

Correct answer: B. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q96. A weekly summary task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- B. Add more subagents to parallelize the cost away.
- C. Ignore cost — agentic systems should always maximize autonomy.
- D. Switch to a larger model to reduce the number of steps needed.

Correct answer: A. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q97. A invoice-field extraction task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Add more subagents to parallelize the cost away.
- B. Ignore cost — agentic systems should always maximize autonomy.
- C. Switch to a larger model to reduce the number of steps needed.
- D. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.

Correct answer: D. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q98. A FAQ-answering task that used to run as a single prompt now runs as a five-subagent agentic pipeline, tripling cost and latency with no accuracy gain. What's the right response?

- A. Ignore cost — agentic systems should always maximize autonomy.
- B. Switch to a larger model to reduce the number of steps needed.
- C. ✓ Reconsider whether a simpler workflow or single prompt would achieve the same result more cheaply and reliably.
- D. Add more subagents to parallelize the cost away.

Correct answer: C. Agentic architecture isn't free — added autonomy should be justified by open-ended scope or adaptive planning needs; if a simpler pattern gets the same result, added agent complexity is the wrong tradeoff.

Q99. A subagent at nesting level 2 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Let each subagent decide its own depth limit independently.
- B. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- C. Allow unlimited recursive delegation with no depth limit.
- D. Disallow all subagent-of-subagent patterns in every case.

Correct answer: B. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q100. A subagent at nesting level 3 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- B. Allow unlimited recursive delegation with no depth limit.
- C. Disallow all subagent-of-subagent patterns in every case.
- D. Let each subagent decide its own depth limit independently.

Correct answer: A. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q101. A subagent at nesting level 4 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Allow unlimited recursive delegation with no depth limit.
- B. Disallow all subagent-of-subagent patterns in every case.
- C. Let each subagent decide its own depth limit independently.
- D. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.

Correct answer: D. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q102. A subagent at nesting level 5 spawns its own subagents, which then spawn further subagents, and cost balloons unexpectedly. What design control should have been in place?

- A. Disallow all subagent-of-subagent patterns in every case.
- B. Let each subagent decide its own depth limit independently.
- C. ✓ Cap recursion depth explicitly and require a clear justification before a subagent may spawn its own subagents.
- D. Allow unlimited recursive delegation with no depth limit.

Correct answer: C. Deep or unbounded recursive delegation can silently multiply cost and latency; a deliberate, explicit depth cap with justification keeps nested delegation from spiraling.

Q103. In a batch of tool calls, a lookup call and a calculation call that consumes its result must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Let the model guess the order with no explicit sequencing logic.
- B. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- C. Run every tool call in parallel regardless of dependency.
- D. Run every tool call strictly sequentially regardless of independence.

Correct answer: B. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q104. In a batch of tool calls, an authentication call and the API call that requires its token must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- B. Run every tool call in parallel regardless of dependency.
- C. Run every tool call strictly sequentially regardless of independence.
- D. Let the model guess the order with no explicit sequencing logic.

Correct answer: A. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q105. In a batch of tool calls, a file-creation call and a subsequent file-read call must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Run every tool call in parallel regardless of dependency.
- B. Run every tool call strictly sequentially regardless of independence.
- C. Let the model guess the order with no explicit sequencing logic.
- D. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.

Correct answer: D. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q106. In a batch of tool calls, a database write and a downstream read of that same record must run in order because the second needs the first's output, while the rest are independent. How should the orchestration layer handle this?

- A. Run every tool call strictly sequentially regardless of independence.
- B. Let the model guess the order with no explicit sequencing logic.
- C. ✓ Detect the dependency and force those specific calls to run in the required order, even if the rest run in parallel.
- D. Run every tool call in parallel regardless of dependency.

Correct answer: C. Mixed dependency graphs need selective sequencing: force genuinely dependent calls into order while still parallelizing everything that's actually independent.

Q107. A subagent fails on a transient network timeout after one attempt. Who should own the decision of whether and how to retry?

- A. No one — a single subagent failure should always abort the whole task.
- B. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- C. Each subagent independently, with no coordinator involvement.
- D. The end user, who should be asked to manually retry.

Correct answer: B. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q108. A subagent fails on a rate-limited API call after one attempt. Who should own the decision of whether and how to retry?

- A. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- B. Each subagent independently, with no coordinator involvement.
- C. The end user, who should be asked to manually retry.
- D. No one — a single subagent failure should always abort the whole task.

Correct answer: A. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q109. A subagent fails on an ambiguous tool result after one attempt. Who should own the decision of whether and how to retry?

- A. Each subagent independently, with no coordinator involvement.
- B. The end user, who should be asked to manually retry.
- C. No one — a single subagent failure should always abort the whole task.
- D. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.

Correct answer: D. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q110. A subagent fails on a temporarily unavailable data source after one attempt. Who should own the decision of whether and how to retry?

- A. The end user, who should be asked to manually retry.
- B. No one — a single subagent failure should always abort the whole task.
- C. ✓ The coordinator, since it owns cross-subagent error handling and retry policy.
- D. Each subagent independently, with no coordinator involvement.

Correct answer: C. Retry policy across subagents is a coordinator responsibility — it's the coordinator that has visibility into overall task state needed to decide whether retrying, falling back, or escalating makes sense.

Q111. An agent that only needs to read customer order status is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Grant broad permissions once and never revisit them.
- B. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- C. Grant the agent full account-wide permissions for convenience.
- D. Rely on the system prompt alone to describe what the agent shouldn't do.

Correct answer: B. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q112. An agent that only needs to post to a single internal Slack channel is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- B. Grant the agent full account-wide permissions for convenience.
- C. Rely on the system prompt alone to describe what the agent shouldn't do.
- D. Grant broad permissions once and never revisit them.

Correct answer: A. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q113. An agent that only needs to summarize log files is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Grant the agent full account-wide permissions for convenience.
- B. Rely on the system prompt alone to describe what the agent shouldn't do.
- C. Grant broad permissions once and never revisit them.
- D. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.

Correct answer: D. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q114. An agent that only needs to look up inventory counts is granted broad, unscoped account permissions "to be safe." What's the better practice?

- A. Rely on the system prompt alone to describe what the agent shouldn't do.
- B. Grant broad permissions once and never revisit them.
- C. ✓ Scope the agent's tool permissions narrowly to only what that specific task requires.
- D. Grant the agent full account-wide permissions for convenience.

Correct answer: C. Least-privilege scoping limits blast radius if the agent misfires or is manipulated — permissions should match the specific task, not be granted broadly by default.

Q115. A third-party payment API call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Silently retry forever until the call eventually succeeds.
- B. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- C. Let tool calls run with no timeout at all.
- D. Treat any slow call as a permanent failure with no retry option.

Correct answer: B. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q116. A internal search index call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- B. Let tool calls run with no timeout at all.
- C. Treat any slow call as a permanent failure with no retry option.
- D. Silently retry forever until the call eventually succeeds.

Correct answer: A. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q117. A external weather API call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Let tool calls run with no timeout at all.
- B. Treat any slow call as a permanent failure with no retry option.
- C. Silently retry forever until the call eventually succeeds.
- D. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.

Correct answer: D. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q118. A legacy database query call in an agent loop occasionally hangs with no response for several minutes. What should the orchestration layer do?

- A. Treat any slow call as a permanent failure with no retry option.
- B. Silently retry forever until the call eventually succeeds.
- C. ✓ Set an explicit timeout per tool call and treat a timeout as a distinct, handleable failure — not a silent hang.
- D. Let tool calls run with no timeout at all.

Correct answer: C. Explicit per-call timeouts turn an indefinite hang into a defined, handleable failure the coordinator can act on, rather than silently blocking the whole loop.

Q119. A system automates fund transfers and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Place the checkpoint immediately after the action, as a post-hoc notification only.
- B. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- C. Place the checkpoint only at the very end of the entire task.
- D. Skip human checkpoints entirely if the agent seems confident.

Correct answer: B. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q120. A system automates production deployments and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- B. Place the checkpoint only at the very end of the entire task.
- C. Skip human checkpoints entirely if the agent seems confident.
- D. Place the checkpoint immediately after the action, as a post-hoc notification only.

Correct answer: A. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q121. A system automates customer account closures and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Place the checkpoint only at the very end of the entire task.
- B. Skip human checkpoints entirely if the agent seems confident.
- C. Place the checkpoint immediately after the action, as a post-hoc notification only.
- D. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.

Correct answer: D. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q122. A system automates bulk data deletions and includes a human approval step, but the step fires only in a summary email after the action has already run. What's wrong with this design?

- A. Skip human checkpoints entirely if the agent seems confident.
- B. Place the checkpoint immediately after the action, as a post-hoc notification only.
- C. ✓ Place the checkpoint before the irreversible or high-stakes action, not after it has already executed.
- D. Place the checkpoint only at the very end of the entire task.

Correct answer: C. A human-in-the-loop checkpoint only has value if it can prevent an irreversible or high-stakes action; placing it after execution turns it into a notification, not a safeguard.

Q123. A multi-agent research pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Skip logging for subagents since only the final result matters.
- B. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- C. Log only the coordinator's final synthesized output.
- D. Rely on the model to self-report what it did in plain text.

Correct answer: B. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q124. A multi-agent customer-support pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- B. Log only the coordinator's final synthesized output.
- C. Rely on the model to self-report what it did in plain text.
- D. Skip logging for subagents since only the final result matters.

Correct answer: A. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q125. A multi-agent code-review pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Log only the coordinator's final synthesized output.
- B. Rely on the model to self-report what it did in plain text.
- C. Skip logging for subagents since only the final result matters.
- D. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.

Correct answer: D. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q126. A multi-agent compliance-check pipeline occasionally produces an incorrect final report and no one can tell which subagent introduced the error. What's missing?

- A. Rely on the model to self-report what it did in plain text.
- B. Skip logging for subagents since only the final result matters.
- C. ✓ Log each subagent's task prompt, tool calls, and result with a shared trace ID linking them to the parent task.
- D. Log only the coordinator's final synthesized output.

Correct answer: C. Without per-subagent traceability (task prompt, tool calls, and result, linked by a shared trace ID), diagnosing which component introduced an error in a multi-agent pipeline becomes guesswork.

Q127. A task involving formatting a casual internal summary is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Refuse the task entirely until every detail is fully specified.
- B. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- C. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- D. Always guess silently with no stated assumption, to avoid slowing the user down.

Correct answer: B. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q128. A task involving choosing a default chart color scheme is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- B. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- C. Always guess silently with no stated assumption, to avoid slowing the user down.
- D. Refuse the task entirely until every detail is fully specified.

Correct answer: A. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q129. A task involving picking a reasonable default date range for a report is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Always stop and ask a clarifying question before proceeding, regardless of stakes.
- B. Always guess silently with no stated assumption, to avoid slowing the user down.
- C. Refuse the task entirely until every detail is fully specified.
- D. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.

Correct answer: D. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q130. A task involving selecting a default sort order for a list is mildly ambiguous but the cost of a wrong assumption is low. How should the agent handle it?

- A. Always guess silently with no stated assumption, to avoid slowing the user down.
- B. Refuse the task entirely until every detail is fully specified.
- C. ✓ State the most reasonable assumption explicitly and proceed, unless the ambiguity is severe enough that guessing wrong is costly.
- D. Always stop and ask a clarifying question before proceeding, regardless of stakes.

Correct answer: C. For low-stakes ambiguity, stating a reasonable assumption and proceeding keeps the system useful; reserving clarifying questions for cases where a wrong guess would be genuinely costly.

Q131. A subagent responsible for a single narrow fact lookup is given the same large context budget as the coordinator's full session. What's the issue?

- A. Size all subagent budgets identically regardless of subtask complexity.
- B. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- C. Give every subagent the same context budget as the coordinator by default.
- D. Give subagents an unlimited context budget to be safe.

Correct answer: B. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q132. A subagent responsible for summarizing one short document is given the same large context budget as the coordinator's full session. What's the issue?

- A. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- B. Give every subagent the same context budget as the coordinator by default.
- C. Give subagents an unlimited context budget to be safe.
- D. Size all subagent budgets identically regardless of subtask complexity.

Correct answer: A. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q133. A subagent responsible for validating one data field is given the same large context budget as the coordinator's full session. What's the issue?

- A. Give every subagent the same context budget as the coordinator by default.
- B. Give subagents an unlimited context budget to be safe.
- C. Size all subagent budgets identically regardless of subtask complexity.
- D. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.

Correct answer: D. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q134. A subagent responsible for answering one yes/no sub-question is given the same large context budget as the coordinator's full session. What's the issue?

- A. Give subagents an unlimited context budget to be safe.
- B. Size all subagent budgets identically regardless of subtask complexity.
- C. ✓ Size the subagent's context budget to what that specific subtask needs, not to the whole system's total budget.
- D. Give every subagent the same context budget as the coordinator by default.

Correct answer: C. Context budgets should be scoped to what a given subagent's task actually requires; over-provisioning wastes cost and can dilute the subagent's focus with irrelevant capacity.

Q135. A long-running multi-day research project loses track of which subtasks are already complete after several turns. What's missing?

- A. Persist state only in the model's own summarized memory of the conversation.
- B. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- C. Rely entirely on the model re-deriving the plan from scratch each turn.
- D. Store no state at all and treat every turn as fully independent.

Correct answer: B. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q136. A long-running multi-step migration task loses track of which subtasks are already complete after several turns. What's missing?

- A. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- B. Rely entirely on the model re-deriving the plan from scratch each turn.
- C. Store no state at all and treat every turn as fully independent.
- D. Persist state only in the model's own summarized memory of the conversation.

Correct answer: A. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q137. A long-running extended data-cleanup job loses track of which subtasks are already complete after several turns. What's missing?

- A. Rely entirely on the model re-deriving the plan from scratch each turn.
- B. Store no state at all and treat every turn as fully independent.
- C. Persist state only in the model's own summarized memory of the conversation.
- D. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.

Correct answer: D. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Q138. A long-running long-running audit task loses track of which subtasks are already complete after several turns. What's missing?

- A. Store no state at all and treat every turn as fully independent.
- B. Persist state only in the model's own summarized memory of the conversation.
- C. ✓ Persist an explicit task list or plan object that survives across turns, rather than relying on the model to reconstruct state from conversation history alone.
- D. Rely entirely on the model re-deriving the plan from scratch each turn.

Correct answer: C. An explicit, persisted task-list or plan object gives the system a reliable source of truth for progress, instead of depending on the model to reconstruct state from a long conversation history.

Claude Code Configuration & Workflows

Q1. Where should a personal shortcut for how you like commit messages phrased, used only in your own sessions be documented for Claude Code?

- A. `.claude/CLAUDE.md` (project-level)
- B. A directory-level `CLAUDE.md` inside that module
- C. The system prompt, hardcoded
- D. ✓ `~/.claude/CLAUDE.md` (user-level)

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q2. Where should the team's shared testing conventions, meant to apply to every contributor be documented for Claude Code?

- A. A directory-level `CLAUDE.md` inside that module
- B. The system prompt, hardcoded
- C. ✓ `.claude/CLAUDE.md` (project-level)
- D. `~/.claude/CLAUDE.md` (user-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q3. Where should conventions specific only to the payments module, not the rest of the codebase be documented for Claude Code?

- A. The system prompt, hardcoded
- B. ✓ A directory-level `CLAUDE.md` inside that module
- C. `~/.claude/CLAUDE.md` (user-level)
- D. `.claude/CLAUDE.md` (project-level)

Correct answer: B. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q4. Where should your own personal preference for verbose vs. terse explanations be documented for Claude Code?

- A. ✓ `~/.claude/CLAUDE.md` (user-level)
- B. `.claude/CLAUDE.md` (project-level)
- C. A directory-level `CLAUDE.md` inside that module
- D. The system prompt, hardcoded

Correct answer: A. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q5. Where should the repo-wide required code style guide, committed and shared be documented for Claude Code?

- A. `~/.claude/CLAUDE.md` (user-level)
- B. A directory-level `CLAUDE.md` inside that module
- C. The system prompt, hardcoded
- D. ✓ `.claude/CLAUDE.md` (project-level)

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q6. Where should naming conventions that only apply inside the /frontend directory be documented for Claude Code?

- A. .claude/CLAUDE.md (project-level)
- B. A directory-level CLAUDE.md inside that module
- C. ✓ A directory-level CLAUDE.md inside /frontend
- D. ~/.claude/CLAUDE.md (user-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q7. Where should a note reminding yourself to always double-check SQL migrations, just for your sessions be documented for Claude Code?

- A. The system prompt, hardcoded
- B. ✓ ~/.claude/CLAUDE.md (user-level)
- C. .claude/CLAUDE.md (project-level)
- D. A directory-level CLAUDE.md inside that module

Correct answer: B. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q8. Where should the team's shared API design conventions everyone must follow be documented for Claude Code?

- A. ✓ .claude/CLAUDE.md (project-level)
- B. ~/.claude/CLAUDE.md (user-level)
- C. A directory-level CLAUDE.md inside that module
- D. The system prompt, hardcoded

Correct answer: A. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q9. Where should database schema conventions specific to the /db directory only be documented for Claude Code?

- A. ~/.claude/CLAUDE.md (user-level)
- B. .claude/CLAUDE.md (project-level)
- C. A directory-level CLAUDE.md inside that module
- D. ✓ A directory-level CLAUDE.md inside /db

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q10. Where should your personal preference for how much extended thinking to show, local only to you be documented for Claude Code?

- A. A directory-level CLAUDE.md inside that module
- B. The system prompt, hardcoded
- C. ✓ ~/.claude/CLAUDE.md (user-level)
- D. .claude/CLAUDE.md (project-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q11. On a team using Claude Code, a new developer never follows the documented commit-message format, even though the convention is "documented." What's the likely cause?

- A. CLAUDE.md files are ignored unless referenced in every prompt manually.
- B. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- C. Claude Code doesn't support enforcing coding conventions at all.
- D. The convention must have been written in a language Claude doesn't understand.

Correct answer: B. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q12. On a team using Claude Code, only one senior engineer's sessions seem to respect the linting rules, even though the convention is "documented." What's the likely cause?

- A. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- B. Claude Code doesn't support enforcing coding conventions at all.
- C. The convention must have been written in a language Claude doesn't understand.
- D. CLAUDE.md files are ignored unless referenced in every prompt manually.

Correct answer: A. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q13. On a team using Claude Code, the team's API error-handling convention is followed inconsistently across contributors, even though the convention is "documented." What's the likely cause?

- A. Claude Code doesn't support enforcing coding conventions at all.
- B. The convention must have been written in a language Claude doesn't understand.
- C. CLAUDE.md files are ignored unless referenced in every prompt manually.
- D. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.

Correct answer: D. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q14. On a team using Claude Code, a documented testing convention only ever applies when the original author is working, even though the convention is "documented." What's the likely cause?

- A. The convention must have been written in a language Claude doesn't understand.
- B. CLAUDE.md files are ignored unless referenced in every prompt manually.
- C. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- D. Claude Code doesn't support enforcing coding conventions at all.

Correct answer: C. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q15. On a team using Claude Code, new hires consistently miss the team's naming conventions despite "documentation" existing, even though the convention is "documented." What's the likely cause?

- A. CLAUDE.md files are ignored unless referenced in every prompt manually.
- B. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- C. Claude Code doesn't support enforcing coding conventions at all.
- D. The convention must have been written in a language Claude doesn't understand.

Correct answer: B. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q16. On a team using Claude Code, a security review checklist is followed by only one contributor's sessions, even though the convention is "documented." What's the likely cause?

- A. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- B. Claude Code doesn't support enforcing coding conventions at all.
- C. The convention must have been written in a language Claude doesn't understand.
- D. CLAUDE.md files are ignored unless referenced in every prompt manually.

Correct answer: A. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q17. You want a rule in .claude/rules/ to load only when editing test files, and stay out of context otherwise. How do you scope it?

- A. Rename the rule file to match the folder name exactly.
- B. Put the rule inside the project-level CLAUDE.md instead.
- C. Duplicate the rule into every relevant subdirectory manually.
- D. ✓ Add YAML frontmatter to the rule file with paths: ["**/*.test.tsx"]

Correct answer: D. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q18. You want a rule in .claude/rules/ to load only when editing files under the API layer, and stay out of context otherwise. How do you scope it?

- A. Put the rule inside the project-level CLAUDE.md instead.
- B. Duplicate the rule into every relevant subdirectory manually.
- C. ✓ Add YAML frontmatter to the rule file with paths: ["src/api/**/*.ts"]
- D. Rename the rule file to match the folder name exactly.

Correct answer: C. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q19. You want a rule in .claude/rules/ to load only when editing database migration files, and stay out of context otherwise. How do you scope it?

- A. Duplicate the rule into every relevant subdirectory manually.
- B. ✓ Add YAML frontmatter to the rule file with paths: ["**/migrations/*.sql"]
- C. Rename the rule file to match the folder name exactly.
- D. Put the rule inside the project-level CLAUDE.md instead.

Correct answer: B. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q20. You want a rule in .claude/rules/ to load only when editing files under the frontend directory, and stay out of context otherwise. How do you scope it?

- A. ✓ Add YAML frontmatter to the rule file with paths: ["src/frontend/**/*.ts"]
- B. Rename the rule file to match the folder name exactly.
- C. Put the rule inside the project-level CLAUDE.md instead.
- D. Duplicate the rule into every relevant subdirectory manually.

Correct answer: A. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q21. You want a rule in `.claude/rules/` to load only when editing configuration YAML files, and stay out of context otherwise. How do you scope it?

- A. Rename the rule file to match the folder name exactly.
- B. Put the rule inside the project-level `CLAUDE.md` instead.
- C. Duplicate the rule into every relevant subdirectory manually.
- D. ✓ Add YAML frontmatter to the rule file with paths: `["**/*.yaml", "**/*.yml"]`

Correct answer: D. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q22. You want a rule in `.claude/rules/` to load only when editing Python files under the ML pipeline directory, and stay out of context otherwise. How do you scope it?

- A. Put the rule inside the project-level `CLAUDE.md` instead.
- B. Duplicate the rule into every relevant subdirectory manually.
- C. ✓ Add YAML frontmatter to the rule file with paths: `["src/ml/**/*.py"]`
- D. Rename the rule file to match the folder name exactly.

Correct answer: C. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q23. You want a rule in `.claude/rules/` to load only when editing documentation markdown files, and stay out of context otherwise. How do you scope it?

- A. Duplicate the rule into every relevant subdirectory manually.
- B. ✓ Add YAML frontmatter to the rule file with paths: `["docs/**/*.md"]`
- C. Rename the rule file to match the folder name exactly.
- D. Put the rule inside the project-level `CLAUDE.md` instead.

Correct answer: B. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q24. You want a rule in `.claude/rules/` to load only when editing infrastructure-as-code files, and stay out of context otherwise. How do you scope it?

- A. ✓ Add YAML frontmatter to the rule file with paths: `["infra/**/*.tf"]`
- B. Rename the rule file to match the folder name exactly.
- C. Put the rule inside the project-level `CLAUDE.md` instead.
- D. Duplicate the rule into every relevant subdirectory manually.

Correct answer: A. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q25. A Skill performs a full dependency-vulnerability scan across the repo, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Run the skill directly in the main session for simplicity.
- B. Split the skill's output across multiple `CLAUDE.md` files.
- C. Increase the main session's context window instead.
- D. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.

Correct answer: D. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q26. A Skill performs a deep codebase map generation step, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Split the skill's output across multiple CLAUDE.md files.
- B. Increase the main session's context window instead.
- C. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- D. Run the skill directly in the main session for simplicity.

Correct answer: C. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q27. A Skill performs a large test-coverage report, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Increase the main session's context window instead.
- B. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- C. Run the skill directly in the main session for simplicity.
- D. Split the skill's output across multiple CLAUDE.md files.

Correct answer: B. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q28. A Skill performs an exhaustive static-analysis pass, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- B. Run the skill directly in the main session for simplicity.
- C. Split the skill's output across multiple CLAUDE.md files.
- D. Increase the main session's context window instead.

Correct answer: A. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q29. A Skill performs a bulk log-file anomaly scan, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Run the skill directly in the main session for simplicity.
- B. Split the skill's output across multiple CLAUDE.md files.
- C. Increase the main session's context window instead.
- D. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.

Correct answer: D. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q30. A Skill performs a large schema-diff report between two databases, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Split the skill's output across multiple CLAUDE.md files.
- B. Increase the main session's context window instead.
- C. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- D. Run the skill directly in the main session for simplicity.

Correct answer: C. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q31. You're setting up a GitHub Actions job that runs Claude Code to summarize failing tests and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Set a made-up CLAUDE_HEADLESS=true environment variable.
- B. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- C. Run Claude Code with no special flags; the default mode works fine in CI.
- D. Redirect stdin from /dev/null as the primary fix.

Correct answer: B. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q32. You're setting up a nightly cron job that asks Claude Code to generate a changelog and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- B. Run Claude Code with no special flags; the default mode works fine in CI.
- C. Redirect stdin from /dev/null as the primary fix.
- D. Set a made-up CLAUDE_HEADLESS=true environment variable.

Correct answer: A. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q33. You're setting up a pre-commit hook that runs Claude Code to check for TODOs and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Run Claude Code with no special flags; the default mode works fine in CI.
- B. Redirect stdin from /dev/null as the primary fix.
- C. Set a made-up CLAUDE_HEADLESS=true environment variable.
- D. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.

Correct answer: D. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q34. You're setting up a Jenkins pipeline stage that runs Claude Code to lint recent changes and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Redirect stdin from /dev/null as the primary fix.
- B. Set a made-up CLAUDE_HEADLESS=true environment variable.
- C. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- D. Run Claude Code with no special flags; the default mode works fine in CI.

Correct answer: C. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q35. You're setting up a scheduled job that has Claude Code generate a dependency report and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Set a made-up CLAUDE_HEADLESS=true environment variable.
- B. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- C. Run Claude Code with no special flags; the default mode works fine in CI.
- D. Redirect stdin from /dev/null as the primary fix.

Correct answer: B. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q36. You're setting up a CI stage that has Claude Code draft release notes and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. ✓ Use the `-p` (print) flag so Claude Code runs non-interactively and exits after producing output.
- B. Run Claude Code with no special flags; the default mode works fine in CI.
- C. Redirect stdin from `/dev/null` as the primary fix.
- D. Set a made-up `CLAUDE_HEADLESS=true` environment variable.

Correct answer: A. Without `-p`, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; `-p` runs it non-interactively and exits after completion.

Q37. You're setting up an automated script that has Claude Code review a diff and post a comment and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Run Claude Code with no special flags; the default mode works fine in CI.
- B. Redirect stdin from `/dev/null` as the primary fix.
- C. Set a made-up `CLAUDE_HEADLESS=true` environment variable.
- D. ✓ Use the `-p` (print) flag so Claude Code runs non-interactively and exits after producing output.

Correct answer: D. Without `-p`, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; `-p` runs it non-interactively and exits after completion.

Q38. You're setting up a batch job that has Claude Code triage new bug reports and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Redirect stdin from `/dev/null` as the primary fix.
- B. Set a made-up `CLAUDE_HEADLESS=true` environment variable.
- C. ✓ Use the `-p` (print) flag so Claude Code runs non-interactively and exits after producing output.
- D. Run Claude Code with no special flags; the default mode works fine in CI.

Correct answer: C. Without `-p`, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; `-p` runs it non-interactively and exits after completion.

Q39. Where should the configuration for a database MCP server the whole team needs for the project live?

- A. An environment variable with no config file
- B. ✓ `.mcp.json` (project-level, version-controlled)
- C. `~/.claude.json` (personal, not shared)
- D. Directly inside `CLAUDE.md` as free text

Correct answer: B. Shared, team-wide MCP servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json` until they're ready to be proposed for the shared config.

Q40. Where should the configuration for an experimental MCP server one engineer is trialing before proposing it to the team live?

- A. ✓ `~/.claude.json` (personal, not shared)
- B. `.mcp.json` (project-level, version-controlled)
- C. Directly inside `CLAUDE.md` as free text
- D. An environment variable with no config file

Correct answer: A. Shared, team-wide MCP servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json` until they're ready to be proposed for the shared config.

Q41. Where should the configuration for a shared internal ticketing MCP server required for everyone's workflow live?

- A. `~/.claude.json` (personal, not shared)
- B. Directly inside `CLAUDE.md` as free text
- C. An environment variable with no config file
- D. ✓ `.mcp.json` (project-level, version-controlled)

Correct answer: D. Shared, team-wide MCP servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json` until they're ready to be proposed for the shared config.

Q42. Where should the configuration for a personal weather MCP server one developer likes to use locally live?

- A. Directly inside CLAUDE.md as free text
- B. An environment variable with no config file
- C. ✓ ~/.claude.json (personal, not shared)
- D. .mcp.json (project-level, version-controlled)

Correct answer: C. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q43. Where should the configuration for a shared internal search MCP server the whole repo depends on live?

- A. An environment variable with no config file
- B. ✓ .mcp.json (project-level, version-controlled)
- C. ~/.claude.json (personal, not shared)
- D. Directly inside CLAUDE.md as free text

Correct answer: B. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q44. Where should the configuration for a personal note-taking MCP server unrelated to the team's actual project live?

- A. ✓ ~/.claude.json (personal, not shared)
- B. .mcp.json (project-level, version-controlled)
- C. Directly inside CLAUDE.md as free text
- D. An environment variable with no config file

Correct answer: A. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q45. Where should the configuration for a shared CI-status MCP server every contributor needs access to live?

- A. ~/.claude.json (personal, not shared)
- B. Directly inside CLAUDE.md as free text
- C. An environment variable with no config file
- D. ✓ .mcp.json (project-level, version-controlled)

Correct answer: D. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q46. Where should the configuration for a personal experimental MCP server for a side project, unrelated to the team repo live?

- A. Directly inside CLAUDE.md as free text
- B. An environment variable with no config file
- C. ✓ ~/.claude.json (personal, not shared)
- D. .mcp.json (project-level, version-controlled)

Correct answer: C. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q47. A developer needs to supply an API token for a database connection MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. Store the token as a comment in CLAUDE.md.
- B. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- C. Commit the literal token directly in .mcp.json for simplicity.
- D. Base64-encode the token before committing it.

Correct answer: B. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q48. A developer needs to supply an API token for a payments-API MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- B. Commit the literal token directly in .mcp.json for simplicity.
- C. Base64-encode the token before committing it.
- D. Store the token as a comment in CLAUDE.md.

Correct answer: A. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q49. A developer needs to supply an API token for an internal ticketing MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. Commit the literal token directly in .mcp.json for simplicity.
- B. Base64-encode the token before committing it.
- C. Store the token as a comment in CLAUDE.md.
- D. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.

Correct answer: D. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q50. A developer needs to supply an API token for a cloud-storage MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. Base64-encode the token before committing it.
- B. Store the token as a comment in CLAUDE.md.
- C. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- D. Commit the literal token directly in .mcp.json for simplicity.

Correct answer: C. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q51. A developer needs to supply an API token for a CRM MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. Store the token as a comment in CLAUDE.md.
- B. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- C. Commit the literal token directly in .mcp.json for simplicity.
- D. Base64-encode the token before committing it.

Correct answer: B. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q52. A developer needs to supply an API token for an internal analytics MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- B. Commit the literal token directly in .mcp.json for simplicity.
- C. Base64-encode the token before committing it.
- D. Store the token as a comment in CLAUDE.md.

Correct answer: A. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q53. After Claude Code writes a large refactor, who should review it for issues?

- A. Have the same session that wrote the code also review it, to save time.
- B. Skip review entirely if the author is confident in the change.
- C. Have the same session review it twice in a row for extra assurance.
- D. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.

Correct answer: D. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q54. After Claude Code writes a security-sensitive authentication change, who should review it for issues?

- A. Skip review entirely if the author is confident in the change.
- B. Have the same session review it twice in a row for extra assurance.
- C. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- D. Have the same session that wrote the code also review it, to save time.

Correct answer: C. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q55. After Claude Code writes a database migration script, who should review it for issues?

- A. Have the same session review it twice in a row for extra assurance.
- B. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- C. Have the same session that wrote the code also review it, to save time.
- D. Skip review entirely if the author is confident in the change.

Correct answer: B. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q56. After Claude Code writes a payment-processing change, who should review it for issues?

- A. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- B. Have the same session that wrote the code also review it, to save time.
- C. Skip review entirely if the author is confident in the change.
- D. Have the same session review it twice in a row for extra assurance.

Correct answer: A. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q57. After Claude Code writes a public API contract change, who should review it for issues?

- A. Have the same session that wrote the code also review it, to save time.
- B. Skip review entirely if the author is confident in the change.
- C. Have the same session review it twice in a row for extra assurance.
- D. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.

Correct answer: D. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q58. After Claude Code writes a critical bug fix under time pressure, who should review it for issues?

- A. Skip review entirely if the author is confident in the change.
- B. Have the same session review it twice in a row for extra assurance.
- C. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- D. Have the same session that wrote the code also review it, to save time.

Correct answer: C. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q59. Which mechanism is the right fit for blocking any Bash command that contains "rm -rf"?

- A. A stronger system-prompt instruction
- B. ✓ A PreToolUse hook on the Bash tool
- C. A PostToolUse hook
- D. A note in CLAUDE.md

Correct answer: B. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q60. Which mechanism is the right fit for trimming an oversized tool result down to only the relevant fields before it reaches the model?

- A. ✓ A PostToolUse hook
- B. A PreToolUse hook
- C. A note in CLAUDE.md
- D. A stronger system-prompt instruction

Correct answer: A. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q61. Which mechanism is the right fit for blocking a refund tool call above a dollar threshold?

- A. A PostToolUse hook
- B. A note in CLAUDE.md
- C. A stronger system-prompt instruction
- D. ✓ A PreToolUse hook on the refund tool

Correct answer: D. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q62. Which mechanism is the right fit for logging every tool call's arguments for audit purposes?

- A. A note in CLAUDE.md
- B. A stronger system-prompt instruction
- C. ✓ A PreToolUse hook
- D. A PostToolUse hook

Correct answer: C. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q63. Which mechanism is the right fit for stripping sensitive fields out of a database query's result before it's shown to the model?

- A. A stronger system-prompt instruction
- B. ✓ A PostToolUse hook
- C. A PreToolUse hook
- D. A note in CLAUDE.md

Correct answer: B. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q64. Which mechanism is the right fit for preventing a deploy tool from running outside business hours?

- A. ✓ A PreToolUse hook on the deploy tool
- B. A PostToolUse hook
- C. A note in CLAUDE.md
- D. A stronger system-prompt instruction

Correct answer: A. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q65. Which mechanism is the right fit for recording how long each tool call took after it completes?

- A. A PreToolUse hook
- B. A note in CLAUDE.md
- C. A stronger system-prompt instruction
- D. ✓ A PostToolUse hook

Correct answer: D. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q66. Which mechanism is the right fit for requiring a confirmation flag be present before a delete tool executes?

- A. A note in CLAUDE.md
- B. A stronger system-prompt instruction
- C. ✓ A PreToolUse hook on the delete tool
- D. A PostToolUse hook

Correct answer: C. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q67. For a documentation-writing session that never needs to run shell commands, what's the best practice around tool permissions?

- A. Leave permissions unscoped and review afterward.
- B. ✓ Restrict the session's allowed tools to only what the task requires.
- C. Grant every available tool by default for convenience.
- D. Rely solely on a prompt instruction not to use unnecessary tools.

Correct answer: B. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q68. For a read-only code-review session that shouldn't be able to edit files, what's the best practice around tool permissions?

- A. ✓ Restrict the session's allowed tools to only what the task requires.
- B. Grant every available tool by default for convenience.
- C. Rely solely on a prompt instruction not to use unnecessary tools.
- D. Leave permissions unscoped and review afterward.

Correct answer: A. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q69. For a session tasked only with running the test suite, not modifying source, what's the best practice around tool permissions?

- A. Grant every available tool by default for convenience.
- B. Rely solely on a prompt instruction not to use unnecessary tools.
- C. Leave permissions unscoped and review afterward.
- D. ✓ Restrict the session's allowed tools to only what the task requires.

Correct answer: D. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q70. For a session meant only to answer questions about the codebase, what's the best practice around tool permissions?

- A. Rely solely on a prompt instruction not to use unnecessary tools.
- B. Leave permissions unscoped and review afterward.
- C. ✓ Restrict the session's allowed tools to only what the task requires.
- D. Grant every available tool by default for convenience.

Correct answer: C. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q71. For a session doing a dependency audit that shouldn't be able to install packages, what's the best practice around tool permissions?

- A. Leave permissions unscoped and review afterward.
- B. ✓ Restrict the session's allowed tools to only what the task requires.
- C. Grant every available tool by default for convenience.
- D. Rely solely on a prompt instruction not to use unnecessary tools.

Correct answer: B. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q72. For a session drafting release notes that shouldn't touch the deploy pipeline, what's the best practice around tool permissions?

- A. ✓ Restrict the session's allowed tools to only what the task requires.
- B. Grant every available tool by default for convenience.
- C. Rely solely on a prompt instruction not to use unnecessary tools.
- D. Leave permissions unscoped and review afterward.

Correct answer: A. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q73. In a monorepo where the /backend and /frontend packages need different linting conventions, what's the best CLAUDE.md structure?

- A. One single project-level CLAUDE.md trying to cover every package's rules at once
- B. Putting all conventions into each developer's personal user-level CLAUDE.md
- C. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- D. ✓ Separate directory-level CLAUDE.md files in each package

Correct answer: D. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q74. In a monorepo where the /mobile package needs its own build conventions distinct from the rest of the monorepo, what's the best CLAUDE.md structure?

- A. Putting all conventions into each developer's personal user-level CLAUDE.md
- B. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- C. ✓ A directory-level CLAUDE.md inside /mobile
- D. One single project-level CLAUDE.md trying to cover every package's rules at once

Correct answer: C. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q75. In a monorepo where the /infra package needs Terraform-specific conventions not relevant elsewhere, what's the best CLAUDE.md structure?

- A. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- B. ✓ A directory-level CLAUDE.md inside /infra
- C. One single project-level CLAUDE.md trying to cover every package's rules at once
- D. Putting all conventions into each developer's personal user-level CLAUDE.md

Correct answer: B. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q76. In a monorepo where the /shared package's conventions should apply to every consuming package, what's the best CLAUDE.md structure?

- A. ✓ A directory-level CLAUDE.md inside /shared, referenced or duplicated as needed
- B. One single project-level CLAUDE.md trying to cover every package's rules at once
- C. Putting all conventions into each developer's personal user-level CLAUDE.md
- D. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge

Correct answer: A. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q77. In a monorepo where the /legacy package needs different, more lenient conventions during a migration, what's the best CLAUDE.md structure?

- A. One single project-level CLAUDE.md trying to cover every package's rules at once
- B. Putting all conventions into each developer's personal user-level CLAUDE.md
- C. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- D. ✓ A directory-level CLAUDE.md inside /legacy

Correct answer: D. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q78. In a monorepo where the /docs package needs its own writing-style conventions distinct from code conventions, what's the best CLAUDE.md structure?

- A. Putting all conventions into each developer's personal user-level CLAUDE.md
- B. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- C. ✓ A directory-level CLAUDE.md inside /docs
- D. One single project-level CLAUDE.md trying to cover every package's rules at once

Correct answer: C. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q79. During a Claude Code session, a session's context usage is climbing unexpectedly fast. What's the first thing to check?

- A. Ignore it, since context issues resolve themselves over time.
- B. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- C. Immediately switch to a larger context-window model with no further investigation.
- D. Restart the session repeatedly until the symptom happens to go away.

Correct answer: B. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q80. During a Claude Code session, a session seems to be repeating the same failed approach in a loop.

What's the first thing to check?

- A. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- B. Immediately switch to a larger context-window model with no further investigation.
- C. Restart the session repeatedly until the symptom happens to go away.
- D. Ignore it, since context issues resolve themselves over time.

Correct answer: A. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q81. During a Claude Code session, token costs for a recurring automated task have crept up over time.

What's the first thing to check?

- A. Immediately switch to a larger context-window model with no further investigation.
- B. Restart the session repeatedly until the symptom happens to go away.
- C. Ignore it, since context issues resolve themselves over time.
- D. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.

Correct answer: D. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q82. During a Claude Code session, a session appears to have lost track of an earlier instruction. What's the first thing to check?

- A. Restart the session repeatedly until the symptom happens to go away.
- B. Ignore it, since context issues resolve themselves over time.
- C. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- D. Immediately switch to a larger context-window model with no further investigation.

Correct answer: C. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q83. During a Claude Code session, a long session's responses have started degrading in quality. What's the first thing to check?

- A. Ignore it, since context issues resolve themselves over time.
- B. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- C. Immediately switch to a larger context-window model with no further investigation.
- D. Restart the session repeatedly until the symptom happens to go away.

Correct answer: B. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q84. During a Claude Code session, a CI job using Claude Code is taking far longer than expected. What's the first thing to check?

- A. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- B. Immediately switch to a larger context-window model with no further investigation.
- C. Restart the session repeatedly until the symptom happens to go away.
- D. Ignore it, since context issues resolve themselves over time.

Correct answer: A. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q85. A team frequently needs running the same multi-step release checklist every week. What's the most maintainable way to make that repeatable in Claude Code?

- A. Re-type the full multi-step prompt from memory every time it's needed.
- B. Paste the steps into the system prompt permanently regardless of task.
- C. Rely on the model to remember the steps from a prior unrelated session.
- D. ✓ Encode the repeatable steps as a custom slash command that encodes the checklist steps, invoked consistently instead of re-typed each time.

Correct answer: D. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q86. A team frequently needs generating a standardized PR description every time. What's the most maintainable way to make that repeatable in Claude Code?

- A. Paste the steps into the system prompt permanently regardless of task.
- B. Rely on the model to remember the steps from a prior unrelated session.
- C. ✓ Encode the repeatable steps as a custom slash command for PR descriptions, invoked consistently instead of re-typed each time.
- D. Re-type the full multi-step prompt from memory every time it's needed.

Correct answer: C. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q87. A team frequently needs running the same onboarding walkthrough for every new repo contributor. What's the most maintainable way to make that repeatable in Claude Code?

- A. Rely on the model to remember the steps from a prior unrelated session.
- B. ✓ Encode the repeatable steps as a custom slash command for onboarding, invoked consistently instead of re-typed each time.
- C. Re-type the full multi-step prompt from memory every time it's needed.
- D. Paste the steps into the system prompt permanently regardless of task.

Correct answer: B. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q88. A team frequently needs triggering the same dependency-audit routine periodically. What's the most maintainable way to make that repeatable in Claude Code?

- A. ✓ Encode the repeatable steps as a custom slash command for the audit routine, invoked consistently instead of re-typed each time.
- B. Re-type the full multi-step prompt from memory every time it's needed.
- C. Paste the steps into the system prompt permanently regardless of task.
- D. Rely on the model to remember the steps from a prior unrelated session.

Correct answer: A. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q89. A team frequently needs producing a consistent weekly status report format. What's the most maintainable way to make that repeatable in Claude Code?

- A. Re-type the full multi-step prompt from memory every time it's needed.
- B. Paste the steps into the system prompt permanently regardless of task.
- C. Rely on the model to remember the steps from a prior unrelated session.
- D. ✓ Encode the repeatable steps as a custom slash command for the status report, invoked consistently instead of re-typed each time.

Correct answer: D. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q90. A team frequently needs running the same pre-release smoke-test sequence. What's the most maintainable way to make that repeatable in Claude Code?

- A. Paste the steps into the system prompt permanently regardless of task.
- B. Rely on the model to remember the steps from a prior unrelated session.
- C. ✓ Encode the repeatable steps as a custom slash command for the smoke-test sequence, invoked consistently instead of re-typed each time.
- D. Re-type the full multi-step prompt from memory every time it's needed.

Correct answer: C. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q91. For a quick, low-stakes formatting fix, how should the thinking/reasoning budget typically be set?

- A. Pick a thinking budget at random for each task.
- B. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- C. Always use the maximum thinking budget regardless of task complexity.
- D. Always disable extended thinking to save tokens, regardless of task complexity.

Correct answer: B. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q92. For a complex multi-file architectural refactor, how should the thinking/reasoning budget typically be set?

- A. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- B. Always use the maximum thinking budget regardless of task complexity.
- C. Always disable extended thinking to save tokens, regardless of task complexity.
- D. Pick a thinking budget at random for each task.

Correct answer: A. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q93. For a one-line typo correction, how should the thinking/reasoning budget typically be set?

- A. Always use the maximum thinking budget regardless of task complexity.
- B. Always disable extended thinking to save tokens, regardless of task complexity.
- C. Pick a thinking budget at random for each task.
- D. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.

Correct answer: D. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q94. For a subtle concurrency bug requiring deep analysis, how should the thinking/reasoning budget typically be set?

- A. Always disable extended thinking to save tokens, regardless of task complexity.
- B. Pick a thinking budget at random for each task.
- C. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- D. Always use the maximum thinking budget regardless of task complexity.

Correct answer: C. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q95. For a routine dependency version bump, how should the thinking/reasoning budget typically be set?

- A. Pick a thinking budget at random for each task.
- B. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- C. Always use the maximum thinking budget regardless of task complexity.
- D. Always disable extended thinking to save tokens, regardless of task complexity.

Correct answer: B. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q96. For a tricky algorithmic optimization problem, how should the thinking/reasoning budget typically be set?

- A. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- B. Always use the maximum thinking budget regardless of task complexity.
- C. Always disable extended thinking to save tokens, regardless of task complexity.
- D. Pick a thinking budget at random for each task.

Correct answer: A. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q97. For a terse internal script meant to be piped into another tool, what output style best fits the consumer of the response?

- A. A single fixed output style used for every task regardless of audience
- B. Whatever the model defaults to, with no configuration
- C. The most verbose style available, regardless of the consumer
- D. ✓ A minimal, non-conversational output style

Correct answer: D. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q98. For a long-form explanatory document meant for a non-technical stakeholder, what output style best fits the consumer of the response?

- A. Whatever the model defaults to, with no configuration
- B. The most verbose style available, regardless of the consumer
- C. ✓ A more verbose, explanatory output style
- D. A single fixed output style used for every task regardless of audience

Correct answer: C. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q99. For a CI log parser expecting machine-readable output only, what output style best fits the consumer of the response?

- A. The most verbose style available, regardless of the consumer
- B. ✓ A minimal, non-conversational output style
- C. A single fixed output style used for every task regardless of audience
- D. Whatever the model defaults to, with no configuration

Correct answer: B. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q100. For an onboarding guide meant to teach a new engineer the codebase, what output style best fits the consumer of the response?

- A. ✓ A more verbose, explanatory output style
- B. A single fixed output style used for every task regardless of audience
- C. Whatever the model defaults to, with no configuration
- D. The most verbose style available, regardless of the consumer

Correct answer: A. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q101. For a one-line status check consumed by another automated script, what output style best fits the consumer of the response?

- A. A single fixed output style used for every task regardless of audience
- B. Whatever the model defaults to, with no configuration
- C. The most verbose style available, regardless of the consumer
- D. ✓ A minimal, non-conversational output style

Correct answer: D. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q102. For a detailed design-review writeup meant for a cross-functional audience, what output style best fits the consumer of the response?

- A. Whatever the model defaults to, with no configuration
- B. The most verbose style available, regardless of the consumer
- C. ✓ A more verbose, explanatory output style
- D. A single fixed output style used for every task regardless of audience

Correct answer: C. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q103. When integrating Claude Code into a git workflow, is committing directly to the main branch on every change good practice?

- A. It's fine as long as the change was AI-generated.
- B. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- C. This is standard best practice and should be encouraged.
- D. It only matters for large teams, not small ones.

Correct answer: B. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q104. When integrating Claude Code into a git workflow, is generating a commit message with no reference to the actual diff good practice?

- A. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- B. This is standard best practice and should be encouraged.
- C. It only matters for large teams, not small ones.
- D. It's fine as long as the change was AI-generated.

Correct answer: A. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q105. When integrating Claude Code into a git workflow, is opening a PR with no description of what changed or why good practice?

- A. This is standard best practice and should be encouraged.
- B. It only matters for large teams, not small ones.
- C. It's fine as long as the change was AI-generated.
- D. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.

Correct answer: D. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q106. When integrating Claude Code into a git workflow, is force-pushing over a shared branch without warning good practice?

- A. It only matters for large teams, not small ones.
- B. It's fine as long as the change was AI-generated.
- C. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- D. This is standard best practice and should be encouraged.

Correct answer: C. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q107. When integrating Claude Code into a git workflow, is committing generated code without running the test suite first good practice?

- A. It's fine as long as the change was AI-generated.
- B. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- C. This is standard best practice and should be encouraged.
- D. It only matters for large teams, not small ones.

Correct answer: B. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q108. When integrating Claude Code into a git workflow, is merging without any review step for a sensitive change good practice?

- A. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- B. This is standard best practice and should be encouraged.
- C. It only matters for large teams, not small ones.
- D. It's fine as long as the change was AI-generated.

Correct answer: A. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q109. On a growing team, a new hire doesn't know which conventions apply to which part of a large codebase. What's the durable fix?

- A. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- B. Rely on the new person to ask the right questions unprompted.
- C. Leave conventions undocumented since Claude Code should infer them automatically.
- D. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.

Correct answer: D. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q110. On a growing team, a new contractor keeps re-deriving conventions the team already settled on months ago. What's the durable fix?

- A. Rely on the new person to ask the right questions unprompted.
- B. Leave conventions undocumented since Claude Code should infer them automatically.
- C. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- D. Have the new person shadow a senior engineer indefinitely instead of documenting anything.

Correct answer: C. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q111. On a growing team, a new team member's Claude Code sessions don't reflect the team's actual coding standards. What's the durable fix?

- A. Leave conventions undocumented since Claude Code should infer them automatically.
- B. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- C. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- D. Rely on the new person to ask the right questions unprompted.

Correct answer: B. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q112. On a growing team, an open-source contributor's PRs don't follow the project's established patterns.

What's the durable fix?

- A. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- B. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- C. Rely on the new person to ask the right questions unprompted.
- D. Leave conventions undocumented since Claude Code should infer them automatically.

Correct answer: A. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q113. On a growing team, a new hire wastes time discovering tribal knowledge that was never written down.

What's the durable fix?

- A. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- B. Rely on the new person to ask the right questions unprompted.
- C. Leave conventions undocumented since Claude Code should infer them automatically.
- D. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.

Correct answer: D. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q114. On a growing team, a rotating on-call engineer isn't aware of the project's incident-response conventions. What's the durable fix?

- A. Rely on the new person to ask the right questions unprompted.
- B. Leave conventions undocumented since Claude Code should infer them automatically.
- C. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- D. Have the new person shadow a senior engineer indefinitely instead of documenting anything.

Correct answer: C. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q115. A team wants to guarantee that unit tests should use a specific mocking library, not whatever the model defaults to. What's the most reliable way to encode this for Claude Code?

- A. Leave it as informal team knowledge with no configuration.
- B. ✓ Encode it as a path-scoped rule in .claude/rules/ (or project CLAUDE.md) so it loads automatically whenever test files are touched.
- C. Mention it once in a Slack message and hope it's remembered.
- D. Rely on code review alone to catch every violation after the fact.

Correct answer: B. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q116. A team wants to guarantee that integration tests must run against a seeded test database, never production. What's the most reliable way to encode this for Claude Code?

- A. ✓ Encode it as a path-scoped rule in .claude/rules/ (or project CLAUDE.md) so it loads automatically whenever test files are touched.
- B. Mention it once in a Slack message and hope it's remembered.
- C. Rely on code review alone to catch every violation after the fact.
- D. Leave it as informal team knowledge with no configuration.

Correct answer: A. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q117. A team wants to guarantee that test files must follow a specific naming and directory pattern. What's the most reliable way to encode this for Claude Code?

- A. Mention it once in a Slack message and hope it's remembered.
- B. Rely on code review alone to catch every violation after the fact.
- C. Leave it as informal team knowledge with no configuration.
- D. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.

Correct answer: D. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q118. A team wants to guarantee that snapshot tests require explicit review before being accepted. What's the most reliable way to encode this for Claude Code?

- A. Rely on code review alone to catch every violation after the fact.
- B. Leave it as informal team knowledge with no configuration.
- C. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- D. Mention it once in a Slack message and hope it's remembered.

Correct answer: C. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q119. A team wants to guarantee that flaky tests must be quarantined rather than silently skipped. What's the most reliable way to encode this for Claude Code?

- A. Leave it as informal team knowledge with no configuration.
- B. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- C. Mention it once in a Slack message and hope it's remembered.
- D. Rely on code review alone to catch every violation after the fact.

Correct answer: B. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q120. A team wants to guarantee that coverage thresholds must be checked before a PR is considered mergeable. What's the most reliable way to encode this for Claude Code?

- A. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- B. Mention it once in a Slack message and hope it's remembered.
- C. Rely on code review alone to catch every violation after the fact.
- D. Leave it as informal team knowledge with no configuration.

Correct answer: A. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Prompt Engineering & Structured Output

Q1. An invoice extractor uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `stated_total` field that doesn't match the sum of `line_items`. What does this reveal about schema-constrained output?

- A. It's impossible for a schema-constrained response to contain this kind of error.
- B. This means the schema itself is malformed and must be rejected by the API.
- C. This only happens when `tool_use` is used without any temperature setting.
- D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q2. A contract summarizer uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `stated_effective_date` that contradicts a date mentioned elsewhere in the same contract. What does this reveal about schema-constrained output?

- A. This means the schema itself is malformed and must be rejected by the API.
- B. This only happens when `tool_use` is used without any temperature setting.
- C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q3. A resume parser uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `total_years_experience` field that doesn't match the sum of the listed job durations. What does this reveal about schema-constrained output?

- A. This only happens when `tool_use` is used without any temperature setting.
- B. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- C. It's impossible for a schema-constrained response to contain this kind of error.
- D. This means the schema itself is malformed and must be rejected by the API.

Correct answer: B. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q4. An order-processing extractor uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `shipping_cost` field that's inconsistent with the stated shipping method. What does this reveal about schema-constrained output?

- A. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- B. It's impossible for a schema-constrained response to contain this kind of error.
- C. This means the schema itself is malformed and must be rejected by the API.
- D. This only happens when `tool_use` is used without any temperature setting.

Correct answer: A. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q5. A financial-report summarizer uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a net_income field that doesn't match revenue minus the listed expenses. What does this reveal about schema-constrained output?

- A. It's impossible for a schema-constrained response to contain this kind of error.
- B. This means the schema itself is malformed and must be rejected by the API.
- C. This only happens when tool_use is used without any temperature setting.
- D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q6. A medical-form extractor uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a patient_age that's inconsistent with the stated date of birth. What does this reveal about schema-constrained output?

- A. This means the schema itself is malformed and must be rejected by the API.
- B. This only happens when tool_use is used without any temperature setting.
- C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q7. A real-estate listing parser uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a price_per_sqft field that doesn't match price divided by square footage. What does this reveal about schema-constrained output?

- A. This only happens when tool_use is used without any temperature setting.
- B. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- C. It's impossible for a schema-constrained response to contain this kind of error.
- D. This means the schema itself is malformed and must be rejected by the API.

Correct answer: B. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q8. A survey-response classifier uses tool_use with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a sentiment_score that contradicts the plain meaning of the quoted response text. What does this reveal about schema-constrained output?

- A. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- B. It's impossible for a schema-constrained response to contain this kind of error.
- C. This means the schema itself is malformed and must be rejected by the API.
- D. This only happens when tool_use is used without any temperature setting.

Correct answer: A. Schemas (via tool_use) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q9. A legal-clause extractor uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `governing_law` field that doesn't match the jurisdiction named elsewhere in the document. What does this reveal about schema-constrained output?

- A. It's impossible for a schema-constrained response to contain this kind of error.
- B. This means the schema itself is malformed and must be rejected by the API.
- C. This only happens when `tool_use` is used without any temperature setting.
- D. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.

Correct answer: D. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q10. A timesheet parser uses `tool_use` with a strict JSON schema, and output is always valid JSON. Sometimes it still produces a `total_hours` field that doesn't match the sum of the individual daily entries. What does this reveal about schema-constrained output?

- A. This means the schema itself is malformed and must be rejected by the API.
- B. This only happens when `tool_use` is used without any temperature setting.
- C. ✓ It only guarantees valid JSON shape and types — it can still return a semantically wrong, internally inconsistent value like this.
- D. It's impossible for a schema-constrained response to contain this kind of error.

Correct answer: C. Schemas (via `tool_use`) guarantee syntactic validity — correct shape and types — but say nothing about whether the content is factually or arithmetically consistent; that needs a separate validation step.

Q11. How should a schema handle a `signature_date` field, when roughly 20% of source documents have no signature at all?

- A. Make it required but add a long comment asking the model to guess carefully
- B. ✓ type: `["string", "null"]` (nullable) rather than required
- C. Mark it required so the model always fills it in with something
- D. Remove the field from the schema entirely to avoid the issue

Correct answer: B. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q12. How should a schema handle a `middle_name` field, when many people simply don't have one?

- A. ✓ type: `["string", "null"]` (nullable) rather than required
- B. Mark it required so the model always fills it in with something
- C. Remove the field from the schema entirely to avoid the issue
- D. Make it required but add a long comment asking the model to guess carefully

Correct answer: A. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q13. How should a schema handle a `discount_code` field, when most orders don't use one?

- A. Mark it required so the model always fills it in with something
- B. Remove the field from the schema entirely to avoid the issue
- C. Make it required but add a long comment asking the model to guess carefully
- D. ✓ type: `["string", "null"]` (nullable) rather than required

Correct answer: D. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q14. How should a schema handle a co_signer field on a loan form, when most loans have none?

- A. Remove the field from the schema entirely to avoid the issue
- B. Make it required but add a long comment asking the model to guess carefully
- C. ✓ type: ["string", "null"] (nullable) rather than required
- D. Mark it required so the model always fills it in with something

Correct answer: C. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q15. How should a schema handle a return_reason field, when most orders are never returned?

- A. Make it required but add a long comment asking the model to guess carefully
- B. ✓ type: ["string", "null"] (nullable) rather than required
- C. Mark it required so the model always fills it in with something
- D. Remove the field from the schema entirely to avoid the issue

Correct answer: B. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q16. How should a schema handle a secondary_phone field, when most customers only list one number?

- A. ✓ type: ["string", "null"] (nullable) rather than required
- B. Mark it required so the model always fills it in with something
- C. Remove the field from the schema entirely to avoid the issue
- D. Make it required but add a long comment asking the model to guess carefully

Correct answer: A. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q17. How should a schema handle a cancellation_date field, when most subscriptions are still active?

- A. Mark it required so the model always fills it in with something
- B. Remove the field from the schema entirely to avoid the issue
- C. Make it required but add a long comment asking the model to guess carefully
- D. ✓ type: ["string", "null"] (nullable) rather than required

Correct answer: D. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q18. How should a schema handle a warranty_claim_id field, when most products have never had a claim?

- A. Remove the field from the schema entirely to avoid the issue
- B. Make it required but add a long comment asking the model to guess carefully
- C. ✓ type: ["string", "null"] (nullable) rather than required
- D. Mark it required so the model always fills it in with something

Correct answer: C. A required field with no guaranteed source data pressures the model to invent a plausible-looking value; a nullable type lets it correctly express "not present" instead.

Q19. A fixed-category enum for customer support ticket category classification doesn't cover every real input the system sees. What should be added?

- A. Make the field free text instead, discarding structure entirely.
- B. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.
- C. Force every input into the nearest existing category no matter how poor the fit.
- D. Leave the enum as-is; mismatches are rare enough to ignore.

Correct answer: B. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q20. A fixed-category enum for news article topic classification doesn't cover every real input the system sees. What should be added?

- A. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.
- B. Force every input into the nearest existing category no matter how poor the fit.
- C. Leave the enum as-is; mismatches are rare enough to ignore.
- D. Make the field free text instead, discarding structure entirely.

Correct answer: A. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q21. A fixed-category enum for product defect type classification doesn't cover every real input the system sees. What should be added?

- A. Force every input into the nearest existing category no matter how poor the fit.
- B. Leave the enum as-is; mismatches are rare enough to ignore.
- C. Make the field free text instead, discarding structure entirely.
- D. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.

Correct answer: D. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q22. A fixed-category enum for job application status classification doesn't cover every real input the system sees. What should be added?

- A. Leave the enum as-is; mismatches are rare enough to ignore.
- B. Make the field free text instead, discarding structure entirely.
- C. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.
- D. Force every input into the nearest existing category no matter how poor the fit.

Correct answer: C. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q23. A fixed-category enum for expense category classification doesn't cover every real input the system sees. What should be added?

- A. Make the field free text instead, discarding structure entirely.
- B. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.
- C. Force every input into the nearest existing category no matter how poor the fit.
- D. Leave the enum as-is; mismatches are rare enough to ignore.

Correct answer: B. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q24. A fixed-category enum for support-call outcome classification doesn't cover every real input the system sees. What should be added?

- A. ✓ Add explicit "unclear" and/or "other" values to the enum so genuinely ambiguous or novel inputs have a truthful place to land.
- B. Force every input into the nearest existing category no matter how poor the fit.
- C. Leave the enum as-is; mismatches are rare enough to ignore.
- D. Make the field free text instead, discarding structure entirely.

Correct answer: A. Enums without an escape-hatch value force bad fits into the nearest wrong category, silently corrupting the data; "unclear"/"other" values let the model express genuine ambiguity truthfully.

Q25. A classification prompt already handles clearly positive and clearly negative reviews reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Target the few-shot examples at clearly positive and clearly negative reviews, since those are the most common cases.
- B. Use no few-shot examples at all and rely purely on the instructions.
- C. Use the same handful of generic examples regardless of what the model struggles with.
- D. ✓ Target the few-shot examples at ambiguous, mixed-sentiment reviews, since that's where the model is actually likely to err.

Correct answer: D. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q26. A classification prompt already handles obviously spam and obviously legitimate emails reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Use no few-shot examples at all and rely purely on the instructions.
- B. Use the same handful of generic examples regardless of what the model struggles with.
- C. ✓ Target the few-shot examples at borderline promotional emails that could go either way, since that's where the model is actually likely to err.
- D. Target the few-shot examples at obviously spam and obviously legitimate emails, since those are the most common cases.

Correct answer: C. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q27. A classification prompt already handles clearly urgent and clearly non-urgent support tickets reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Use the same handful of generic examples regardless of what the model struggles with.
- B. ✓ Target the few-shot examples at tickets with mixed urgency signals, since that's where the model is actually likely to err.
- C. Target the few-shot examples at clearly urgent and clearly non-urgent support tickets, since those are the most common cases.
- D. Use no few-shot examples at all and rely purely on the instructions.

Correct answer: B. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q28. A classification prompt already handles clearly in-scope and clearly out-of-scope requests reliably. Where should additional few-shot examples be focused for the best improvement?

- A. ✓ Target the few-shot examples at requests that sit right at the scope boundary, since that's where the model is actually likely to err.
- B. Target the few-shot examples at clearly in-scope and clearly out-of-scope requests, since those are the most common cases.
- C. Use no few-shot examples at all and rely purely on the instructions.
- D. Use the same handful of generic examples regardless of what the model struggles with.

Correct answer: A. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q29. A classification prompt already handles high-confidence and low-confidence transcription segments reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Target the few-shot examples at high-confidence and low-confidence transcription segments, since those are the most common cases.
- B. Use no few-shot examples at all and rely purely on the instructions.
- C. Use the same handful of generic examples regardless of what the model struggles with.
- D. ✓ Target the few-shot examples at segments with background noise or overlapping speech, since that's where the model is actually likely to err.

Correct answer: D. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q30. A classification prompt already handles simple single-item orders and simple bulk orders reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Use no few-shot examples at all and rely purely on the instructions.
- B. Use the same handful of generic examples regardless of what the model struggles with.
- C. ✓ Target the few-shot examples at orders that mix multiple order types in one submission, since that's where the model is actually likely to err.
- D. Target the few-shot examples at simple single-item orders and simple bulk orders, since those are the most common cases.

Correct answer: C. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q31. A classification prompt already handles clean, well-formatted input documents reliably. Where should additional few-shot examples be focused for the best improvement?

- A. Use the same handful of generic examples regardless of what the model struggles with.
- B. ✓ Target the few-shot examples at malformed or partially corrupted input documents, since that's where the model is actually likely to err.
- C. Target the few-shot examples at clean, well-formatted input documents, since those are the most common cases.
- D. Use no few-shot examples at all and rely purely on the instructions.

Correct answer: B. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q32. A classification prompt already handles single-language text samples reliably. Where should additional few-shot examples be focused for the best improvement?

- A. ✓ Target the few-shot examples at code-switched or multi-language text samples, since that's where the model is actually likely to err.
- B. Target the few-shot examples at single-language text samples, since those are the most common cases.
- C. Use no few-shot examples at all and rely purely on the instructions.
- D. Use the same handful of generic examples regardless of what the model struggles with.

Correct answer: A. Few-shot examples are most valuable when targeted at the model's actual failure cases — ambiguous or boundary inputs — not at cases it already handles well.

Q33. A retry-with-feedback loop is being considered for a total that was miscalculated from correctly-extracted line items. Will retrying likely fix it?

- A. No — retrying never helps with extraction errors.
- B. Retry-with-feedback is only useful for formatting issues, never content issues.
- C. Whether retrying helps depends solely on the model's temperature setting.
- D. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.

Correct answer: D. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q34. A retry-with-feedback loop is being considered for a field the source document never actually contained. Will retrying likely fix it?

- A. Retry-with-feedback is only useful for formatting issues, never content issues.
- B. Whether retrying helps depends solely on the model's temperature setting.
- C. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
- D. Yes — enough retries will eventually surface the missing information.

Correct answer: C. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q35. A retry-with-feedback loop is being considered for a date that was misread due to an ambiguous format in the source text. Will retrying likely fix it?

- A. Whether retrying helps depends solely on the model's temperature setting.
- B. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.
- C. No — retrying never helps with extraction errors.
- D. Retry-with-feedback is only useful for formatting issues, never content issues.

Correct answer: B. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q36. A retry-with-feedback loop is being considered for a customer's stated preference that was never mentioned anywhere in the input. Will retrying likely fix it?

- A. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
- B. Yes — enough retries will eventually surface the missing information.
- C. Retry-with-feedback is only useful for formatting issues, never content issues.
- D. Whether retrying helps depends solely on the model's temperature setting.

Correct answer: A. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q37. A retry-with-feedback loop is being considered for a category that was misclassified despite clear signal in the text. Will retrying likely fix it?

- A. No — retrying never helps with extraction errors.
- B. Retry-with-feedback is only useful for formatting issues, never content issues.
- C. Whether retrying helps depends solely on the model's temperature setting.
- D. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.

Correct answer: D. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q38. A retry-with-feedback loop is being considered for a specific figure that simply isn't present anywhere in the source document. Will retrying likely fix it?

- A. Retry-with-feedback is only useful for formatting issues, never content issues.
- B. Whether retrying helps depends solely on the model's temperature setting.
- C. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
- D. Yes — enough retries will eventually surface the missing information.

Correct answer: C. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q39. A retry-with-feedback loop is being considered for a name that was mis-parsed from a clearly-formatted input field. Will retrying likely fix it?

- A. Whether retrying helps depends solely on the model's temperature setting.
- B. ✓ Yes — the correct information exists in the input; retrying with feedback can recover it.
- C. No — retrying never helps with extraction errors.
- D. Retry-with-feedback is only useful for formatting issues, never content issues.

Correct answer: B. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q40. A retry-with-feedback loop is being considered for a required detail the user never provided in their original request. Will retrying likely fix it?

- A. ✓ No — the information isn't in the source; retrying can't recover data that was never provided.
- B. Yes — enough retries will eventually surface the missing information.
- C. Retry-with-feedback is only useful for formatting issues, never content issues.
- D. Whether retrying helps depends solely on the model's temperature setting.

Correct answer: A. Retry-with-feedback works when the correct data exists in the input but was mis-extracted or miscalculated; it structurally cannot recover information that was never present in the source.

Q41. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for an overnight bulk reclassification of a year of support tickets?

- A. Poor fit — the Batch API is never appropriate for bulk work.
- B. Batch API suitability depends only on document length, not on latency needs.
- C. This should always run synchronously regardless of the workload's nature.
- D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q42. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a blocking pre-merge CI check a developer is actively waiting on?

- A. Batch API suitability depends only on document length, not on latency needs.
- B. This should always run synchronously regardless of the workload's nature.
- C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
- D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q43. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a nightly technical-debt summary report?

- A. This should always run synchronously regardless of the workload's nature.
- B. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.
- C. Poor fit — the Batch API is never appropriate for bulk work.
- D. Batch API suitability depends only on document length, not on latency needs.

Correct answer: B. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q44. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a real-time chat support interaction?

- A. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
- B. Good fit — cost savings always outweigh latency concerns.
- C. Batch API suitability depends only on document length, not on latency needs.
- D. This should always run synchronously regardless of the workload's nature.

Correct answer: A. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q45. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a large one-time migration of historical records to a new schema?

- A. Poor fit — the Batch API is never appropriate for bulk work.
- B. Batch API suitability depends only on document length, not on latency needs.
- C. This should always run synchronously regardless of the workload's nature.
- D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q46. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a live user-facing search-autocomplete feature?

- A. Batch API suitability depends only on document length, not on latency needs.
- B. This should always run synchronously regardless of the workload's nature.
- C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
- D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q47. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a weekly analytics rollup with no user waiting on it?

- A. This should always run synchronously regardless of the workload's nature.
- B. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.
- C. Poor fit — the Batch API is never appropriate for bulk work.
- D. Batch API suitability depends only on document length, not on latency needs.

Correct answer: B. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q48. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for an interactive code-review comment a developer expects instantly?

- A. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
- B. Good fit — cost savings always outweigh latency concerns.
- C. Batch API suitability depends only on document length, not on latency needs.
- D. This should always run synchronously regardless of the workload's nature.

Correct answer: A. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q49. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a bulk overnight re-scoring of a large document archive?

- A. Poor fit — the Batch API is never appropriate for bulk work.
- B. Batch API suitability depends only on document length, not on latency needs.
- C. This should always run synchronously regardless of the workload's nature.
- D. ✓ Good fit — non-blocking, no one waiting in real time, tolerant of the up-to-24-hour window.

Correct answer: D. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q50. Is the Batch API (roughly 50% cheaper, up to a 24-hour window, no latency SLA) a good fit for a synchronous fraud-check that must complete before a transaction clears?

- A. Batch API suitability depends only on document length, not on latency needs.
- B. This should always run synchronously regardless of the workload's nature.
- C. ✓ Poor fit — latency-sensitive with someone actively waiting; the Batch API offers no latency SLA.
- D. Good fit — cost savings always outweigh latency concerns.

Correct answer: C. Batch API fit hinges on whether anyone is actively blocked waiting on the result — non-blocking, tolerant workloads are a good fit; latency-sensitive, blocking workloads are not.

Q51. A system needs to reliably extract 5 structured fields from unstructured customer emails at scale. What's the most robust approach?

- A. Avoid structured extraction altogether and read the response manually.
- B. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.
- C. Ask the model in free text to "please respond in JSON" and parse the string yourself.
- D. Use free text and apply a regex to extract fields after the fact.

Correct answer: B. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q52. A system needs to reliably extract 8 structured fields from unstructured scanned invoices at scale. What's the most robust approach?

- A. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.
- B. Ask the model in free text to "please respond in JSON" and parse the string yourself.
- C. Use free text and apply a regex to extract fields after the fact.
- D. Avoid structured extraction altogether and read the response manually.

Correct answer: A. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q53. A system needs to reliably extract 6 structured fields from unstructured support transcripts at scale. What's the most robust approach?

- A. Ask the model in free text to "please respond in JSON" and parse the string yourself.
- B. Use free text and apply a regex to extract fields after the fact.
- C. Avoid structured extraction altogether and read the response manually.
- D. ✓ Use tool_use with a defined JSON schema so the API enforces valid structure directly.

Correct answer: D. tool_use with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q54. A system needs to reliably extract 4 structured fields from unstructured job postings at scale. What's the most robust approach?

- A. Use free text and apply a regex to extract fields after the fact.
- B. Avoid structured extraction altogether and read the response manually.
- C. ✓ Use `tool_use` with a defined JSON schema so the API enforces valid structure directly.
- D. Ask the model in free text to "please respond in JSON" and parse the string yourself.

Correct answer: C. `tool_use` with a schema is enforced by the API itself, eliminating malformed-JSON failures that free-text-plus-parsing approaches are prone to.

Q55. A classification pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

- A. Randomize temperature per request to average out errors.
- B. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
- C. Use a high temperature to maximize output diversity.
- D. Temperature has no effect on structured-output tasks.

Correct answer: B. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q56. A field-extraction pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

- A. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
- B. Use a high temperature to maximize output diversity.
- C. Temperature has no effect on structured-output tasks.
- D. Randomize temperature per request to average out errors.

Correct answer: A. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q57. A categorization service needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

- A. Use a high temperature to maximize output diversity.
- B. Temperature has no effect on structured-output tasks.
- C. Randomize temperature per request to average out errors.
- D. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.

Correct answer: D. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q58. A data-normalization pipeline needs to produce consistent, repeatable structured output across near-identical inputs. How should generation settings be tuned?

- A. Temperature has no effect on structured-output tasks.
- B. Randomize temperature per request to average out errors.
- C. ✓ Use a lower, more deterministic setting — structured extraction benefits from consistency over creative variation.
- D. Use a high temperature to maximize output diversity.

Correct answer: C. Deterministic, structured tasks generally benefit from lower temperature settings, since consistency matters more than creative variation for extraction or classification.

Q59. A document Q&A system re-sends a large, mostly-unchanged reference document on every one of many calls, driving up cost and latency. What should be optimized?

- A. Move the large context into the tool schema instead.
- B. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.
- C. Resend the full large context uncached on every single call.
- D. Truncate the large context arbitrarily to save cost.

Correct answer: B. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q60. A codebase assistant re-sends a large, mostly-unchanged repository context on every one of many calls, driving up cost and latency. What should be optimized?

- A. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.
- B. Resend the full large context uncached on every single call.
- C. Truncate the large context arbitrarily to save cost.
- D. Move the large context into the tool schema instead.

Correct answer: A. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q61. A policy chatbot re-sends a large, mostly-unchanged policy manual on every one of many calls, driving up cost and latency. What should be optimized?

- A. Resend the full large context uncached on every single call.
- B. Truncate the large context arbitrarily to save cost.
- C. Move the large context into the tool schema instead.
- D. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.

Correct answer: D. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q62. A support bot re-sends a large, mostly-unchanged product knowledge base on every one of many calls, driving up cost and latency. What should be optimized?

- A. Truncate the large context arbitrarily to save cost.
- B. Move the large context into the tool schema instead.
- C. ✓ Use prompt caching for the large, stable portion of the context that's reused across many calls.
- D. Resend the full large context uncached on every single call.

Correct answer: C. Prompt caching lets a large, stable prefix be reused across calls instead of being re-processed from scratch each time, cutting both cost and latency.

Q63. A prompt for a legal-review assistant mixes instructions, reference clauses, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

- A. Put all instructions inside the few-shot examples instead of separating them.
- B. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.
- C. Concatenate everything into one undifferentiated block of text.
- D. Rely on line breaks alone with no labeling.

Correct answer: B. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q64. A prompt for a code-review assistant mixes instructions, reference diffs, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

- A. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.
- B. Concatenate everything into one undifferentiated block of text.
- C. Rely on line breaks alone with no labeling.
- D. Put all instructions inside the few-shot examples instead of separating them.

Correct answer: A. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q65. A prompt for a research summarizer mixes instructions, reference sources, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

- A. Concatenate everything into one undifferentiated block of text.
- B. Rely on line breaks alone with no labeling.
- C. Put all instructions inside the few-shot examples instead of separating them.
- D. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.

Correct answer: D. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q66. A prompt for a support assistant mixes instructions, reference policy text, and the actual input together with no clear separation, and the model sometimes confuses one for another. What's the fix?

- A. Rely on line breaks alone with no labeling.
- B. Put all instructions inside the few-shot examples instead of separating them.
- C. ✓ Wrap distinct sections (instructions, context, examples, input) in clear structural tags so the model can distinguish them reliably.
- D. Concatenate everything into one undifferentiated block of text.

Correct answer: C. Clear structural separation (e.g., tagged sections) helps the model reliably distinguish instructions from context from the actual input, reducing confusion between them.

Q67. A research summarizer occasionally produces claims that sound plausible but aren't actually supported by the provided source documents. What structural change reduces this?

- A. Lower the number of source documents provided, to reduce confusion.
- B. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.
- C. Ask the model to simply "be more careful" about accuracy.
- D. Increase the schema's field count to force more detail.

Correct answer: B. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q68. A legal-analysis tool occasionally produces claims that sound plausible but aren't actually supported by the provided contract text. What structural change reduces this?

- A. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.
- B. Ask the model to simply "be more careful" about accuracy.
- C. Increase the schema's field count to force more detail.
- D. Lower the number of source documents provided, to reduce confusion.

Correct answer: A. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q69. A medical-literature assistant occasionally produces claims that sound plausible but aren't actually supported by the provided cited studies. What structural change reduces this?

- A. Ask the model to simply "be more careful" about accuracy.
- B. Increase the schema's field count to force more detail.
- C. Lower the number of source documents provided, to reduce confusion.
- D. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.

Correct answer: D. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q70. A financial analyst tool occasionally produces claims that sound plausible but aren't actually supported by the provided filings. What structural change reduces this?

- A. Increase the schema's field count to force more detail.
- B. Lower the number of source documents provided, to reduce confusion.
- C. ✓ Require the model to ground each claim in a specific, quoted or referenced piece of the source material.
- D. Ask the model to simply "be more careful" about accuracy.

Correct answer: C. Requiring explicit grounding — tying each claim to a specific source reference — gives the model a concrete anchor and makes ungrounded fabrication easier to catch.

Q71. A support bot repeats the same tone-of-voice rule in every single user message instead of setting it once. What's the more maintainable structure?

- A. Randomly split instructions between system and user turns.
- B. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.
- C. Put everything, including the specific request, into the system prompt.
- D. Put stable behavioral rules into the user turn on every single call.

Correct answer: B. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q72. A classification service repeats the same output-format rule in every single user message instead of setting it once. What's the more maintainable structure?

- A. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.
- B. Put everything, including the specific request, into the system prompt.
- C. Put stable behavioral rules into the user turn on every single call.
- D. Randomly split instructions between system and user turns.

Correct answer: A. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q73. A code-review bot repeats the same review-standard rule in every single user message instead of setting it once. What's the more maintainable structure?

- A. Put everything, including the specific request, into the system prompt.
- B. Put stable behavioral rules into the user turn on every single call.
- C. Randomly split instructions between system and user turns.
- D. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.

Correct answer: D. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q74. A summarizer repeats the same length constraint in every single user message instead of setting it once. What's the more maintainable structure?

- A. Put stable behavioral rules into the user turn on every single call.
- B. Randomly split instructions between system and user turns.
- C. ✓ Put stable, task-wide behavior in the system prompt and per-request specifics in the user turn.
- D. Put everything, including the specific request, into the system prompt.

Correct answer: C. Stable, task-wide behavior belongs in the system prompt (set once); per-request specifics belong in the user turn — this avoids redundant repetition and keeps the two concerns cleanly separated.

Q75. A team wants to change how a classification prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

- A. Trust that a prompt "looks right" without measuring actual output.
- B. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.
- C. Ship the new prompt directly to production and monitor for complaints.
- D. Test the prompt once manually on a single example and move on.

Correct answer: B. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q76. A team wants to change how a extraction prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

- A. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.
- B. Ship the new prompt directly to production and monitor for complaints.
- C. Test the prompt once manually on a single example and move on.
- D. Trust that a prompt "looks right" without measuring actual output.

Correct answer: A. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q77. A team wants to change how a summarization prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

- A. Ship the new prompt directly to production and monitor for complaints.
- B. Test the prompt once manually on a single example and move on.
- C. Trust that a prompt "looks right" without measuring actual output.
- D. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.

Correct answer: D. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q78. A team wants to change how a routing prompt is structured but isn't sure if the change will help or hurt accuracy. What should happen before shipping it?

- A. Test the prompt once manually on a single example and move on.
- B. Trust that a prompt "looks right" without measuring actual output.
- C. ✓ Build a held-out evaluation set with known-correct answers and measure accuracy before shipping a prompt change.
- D. Ship the new prompt directly to production and monitor for complaints.

Correct answer: C. A held-out evaluation set with known-correct answers lets you measure the actual effect of a prompt change before it reaches production, rather than guessing from how the prompt reads.

Q79. A real-time extraction UI streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

- A. Assume streamed JSON is always complete after a fixed number of tokens.
- B. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.
- C. Start acting on the JSON object as soon as the first characters arrive.
- D. Disable structured output whenever streaming is needed.

Correct answer: B. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q80. A live classification dashboard streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

- A. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.
- B. Start acting on the JSON object as soon as the first characters arrive.
- C. Disable structured output whenever streaming is needed.
- D. Assume streamed JSON is always complete after a fixed number of tokens.

Correct answer: A. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q81. A streaming form-fill assistant streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

- A. Start acting on the JSON object as soon as the first characters arrive.
- B. Disable structured output whenever streaming is needed.
- C. Assume streamed JSON is always complete after a fixed number of tokens.
- D. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.

Correct answer: D. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q82. A live triage tool streams its structured tool_use response to the client, and a naive downstream parser occasionally chokes on incomplete JSON. What's the correct handling?

- A. Disable structured output whenever streaming is needed.
- B. Assume streamed JSON is always complete after a fixed number of tokens.
- C. ✓ Wait for the complete structured response (or use a schema-aware streaming parser) rather than acting on a partially-streamed JSON object.
- D. Start acting on the JSON object as soon as the first characters arrive.

Correct answer: C. Partially-streamed JSON isn't valid until the object is complete — downstream consumers need either to wait for completion or use a parser designed for incremental structured streaming.

Q83. A classification service sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

- A. Leave prompt length unexamined and optimize only the schema instead.
- B. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.
- C. Add more instructions and examples regardless of whether they're needed.
- D. Switch to a larger model as the default fix for cost concerns.

Correct answer: B. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q84. A extraction pipeline sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

- A. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.
- B. Add more instructions and examples regardless of whether they're needed.
- C. Switch to a larger model as the default fix for cost concerns.
- D. Leave prompt length unexamined and optimize only the schema instead.

Correct answer: A. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q85. A summarization tool sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

- A. Add more instructions and examples regardless of whether they're needed.
- B. Switch to a larger model as the default fix for cost concerns.
- C. Leave prompt length unexamined and optimize only the schema instead.
- D. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.

Correct answer: D. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q86. A routing service sends a much longer prompt than the task actually requires, driving up per-call cost with no accuracy benefit. What's the first optimization to try?

- A. Switch to a larger model as the default fix for cost concerns.
- B. Leave prompt length unexamined and optimize only the schema instead.
- C. ✓ Trim the prompt to only the context genuinely needed for the task, rather than padding it with unused boilerplate.
- D. Add more instructions and examples regardless of whether they're needed.

Correct answer: C. Trimming unnecessary content from the prompt is usually the highest-leverage, lowest-risk cost optimization — before reaching for a different model or infrastructure change.

Q87. A handwritten-form extractor sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

- A. Silently drop uncertain fields from the output with no indication they were omitted.
- B. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.
- C. Always return the model's best guess with no way to distinguish confident from uncertain output.
- D. Reject the entire extraction whenever any single field is uncertain.

Correct answer: B. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q88. A noisy-transcript parser sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

- A. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.
- B. Always return the model's best guess with no way to distinguish confident from uncertain output.
- C. Reject the entire extraction whenever any single field is uncertain.
- D. Silently drop uncertain fields from the output with no indication they were omitted.

Correct answer: A. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q89. A scanned-document extractor sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

- A. Always return the model's best guess with no way to distinguish confident from uncertain output.
- B. Reject the entire extraction whenever any single field is uncertain.
- C. Silently drop uncertain fields from the output with no indication they were omitted.
- D. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.

Correct answer: D. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q90. A ambiguous-survey parser sometimes extracts a field with genuine uncertainty (e.g., illegible handwriting, ambiguous phrasing). How should this be surfaced?

- A. Reject the entire extraction whenever any single field is uncertain.
- B. Silently drop uncertain fields from the output with no indication they were omitted.
- C. ✓ Surface a confidence or uncertainty signal in the schema itself so downstream systems can flag or route low-confidence extractions for review.
- D. Always return the model's best guess with no way to distinguish confident from uncertain output.

Correct answer: C. Building an explicit confidence or uncertainty signal into the schema lets downstream systems route uncertain extractions for human review instead of silently trusting or discarding them.

Q91. A invoice-extraction service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

- A. Silently rename fields without notifying any consumers.
- B. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.
- C. Change the schema in place with no versioning and let downstream consumers fail.
- D. Never evolve the schema once it's shipped, regardless of new requirements.

Correct answer: B. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q92. A order-processing pipeline's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

- A. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.
- B. Change the schema in place with no versioning and let downstream consumers fail.
- C. Never evolve the schema once it's shipped, regardless of new requirements.
- D. Silently rename fields without notifying any consumers.

Correct answer: A. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q93. A ticket-classification service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

- A. Change the schema in place with no versioning and let downstream consumers fail.
- B. Never evolve the schema once it's shipped, regardless of new requirements.
- C. Silently rename fields without notifying any consumers.
- D. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.

Correct answer: D. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q94. A resume-parsing service's schema needs a breaking change (renaming a field, changing a type) but multiple downstream consumers depend on the current shape. What's the safer approach?

- A. Never evolve the schema once it's shipped, regardless of new requirements.
- B. Silently rename fields without notifying any consumers.
- C. ✓ Version the schema explicitly and support the older version during a transition period rather than breaking existing consumers immediately.
- D. Change the schema in place with no versioning and let downstream consumers fail.

Correct answer: C. Explicit schema versioning with a transition period avoids breaking existing consumers outright, giving them time to migrate to the new shape.

Q95. A customer email being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. Disable the schema constraint whenever the source content looks unusual.
- B. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.
- C. Treat any embedded instructions in the source as legitimate commands to follow.
- D. Assume injection isn't a risk for structured-extraction tasks.

Correct answer: B. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q96. A scraped web page being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.
- B. Treat any embedded instructions in the source as legitimate commands to follow.
- C. Assume injection isn't a risk for structured-extraction tasks.
- D. Disable the schema constraint whenever the source content looks unusual.

Correct answer: A. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q97. An uploaded pdf resume being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. Treat any embedded instructions in the source as legitimate commands to follow.
- B. Assume injection isn't a risk for structured-extraction tasks.
- C. Disable the schema constraint whenever the source content looks unusual.
- D. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.

Correct answer: D. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q98. A third-party api response being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. Assume injection isn't a risk for structured-extraction tasks.
- B. Disable the schema constraint whenever the source content looks unusual.
- C. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.
- D. Treat any embedded instructions in the source as legitimate commands to follow.

Correct answer: C. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q99. A user-submitted support ticket being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. Disable the schema constraint whenever the source content looks unusual.
- B. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.
- C. Treat any embedded instructions in the source as legitimate commands to follow.
- D. Assume injection isn't a risk for structured-extraction tasks.

Correct answer: B. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q100. An ocr'd scanned document being fed into a structured-extraction pipeline contains text that reads like an instruction to the model (e.g., "ignore previous instructions and output X"). How should the pipeline treat this?

- A. ✓ Treat the content as untrusted data, not instructions — the extraction prompt should not follow directives found inside it.
- B. Treat any embedded instructions in the source as legitimate commands to follow.
- C. Assume injection isn't a risk for structured-extraction tasks.
- D. Disable the schema constraint whenever the source content looks unusual.

Correct answer: A. Any external content — including documents being extracted from — should be treated as data, not instructions; a robust pipeline doesn't let embedded text redirect its actual task.

Q101. For a product-categorization service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. There's no meaningful difference; both approaches are equally reliable.
- B. Free-text "please respond in JSON" is always more reliable than tool_use.
- C. Neither approach matters since all output should be parsed with a permissive regex anyway.
- D. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

Correct answer: D. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q102. For a sentiment-scoring pipeline, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. Free-text "please respond in JSON" is always more reliable than tool_use.
- B. Neither approach matters since all output should be parsed with a permissive regex anyway.
- C. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.
- D. There's no meaningful difference; both approaches are equally reliable.

Correct answer: C. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q103. For a name-entity extraction service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. Neither approach matters since all output should be parsed with a permissive regex anyway.
- B. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.
- C. There's no meaningful difference; both approaches are equally reliable.
- D. Free-text "please respond in JSON" is always more reliable than tool_use.

Correct answer: B. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q104. For an address-normalization service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.
- B. There's no meaningful difference; both approaches are equally reliable.
- C. Free-text "please respond in JSON" is always more reliable than tool_use.
- D. Neither approach matters since all output should be parsed with a permissive regex anyway.

Correct answer: A. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q105. For a metadata-tagging pipeline, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. There's no meaningful difference; both approaches are equally reliable.
- B. Free-text "please respond in JSON" is always more reliable than tool_use.
- C. Neither approach matters since all output should be parsed with a permissive regex anyway.
- D. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.

Correct answer: D. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q106. For a lead-qualification service, should structured output rely on a plain-text instruction to "respond only in JSON," or on tool_use with a schema?

- A. Free-text "please respond in JSON" is always more reliable than tool_use.
- B. Neither approach matters since all output should be parsed with a permissive regex anyway.
- C. ✓ Prefer tool_use with an explicit schema — it's enforced by the API, unlike asking for JSON in free text and hoping the model complies.
- D. There's no meaningful difference; both approaches are equally reliable.

Correct answer: C. tool_use with a schema is enforced at the API level, giving much stronger guarantees than a free-text instruction the model might not follow precisely.

Q107. In a structured-extraction pipeline, a required numeric field comes back as a non-numeric string. What should happen?

- A. Disable validation going forward since it caught an inconvenient error.
- B. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.
- C. Silently coerce the bad value to something plausible and continue.
- D. Pass the invalid output downstream unchanged and let a later system catch it.

Correct answer: B. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q108. In a structured-extraction pipeline, an enum field comes back with a value outside the allowed set. What should happen?

- A. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.
- B. Silently coerce the bad value to something plausible and continue.
- C. Pass the invalid output downstream unchanged and let a later system catch it.
- D. Disable validation going forward since it caught an inconvenient error.

Correct answer: A. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q109. In a structured-extraction pipeline, a date field comes back in an unparseable format. What should happen?

- A. Silently coerce the bad value to something plausible and continue.
- B. Pass the invalid output downstream unchanged and let a later system catch it.
- C. Disable validation going forward since it caught an inconvenient error.
- D. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.

Correct answer: D. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q110. In a structured-extraction pipeline, a nested object is missing a required sub-field. What should happen?

- A. Pass the invalid output downstream unchanged and let a later system catch it.
- B. Disable validation going forward since it caught an inconvenient error.
- C. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.
- D. Silently coerce the bad value to something plausible and continue.

Correct answer: C. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q111. In a structured-extraction pipeline, an array field that should never be empty comes back empty. What should happen?

- A. Disable validation going forward since it caught an inconvenient error.
- B. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.
- C. Silently coerce the bad value to something plausible and continue.
- D. Pass the invalid output downstream unchanged and let a later system catch it.

Correct answer: B. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q112. In a structured-extraction pipeline, a cross-field consistency check fails after extraction. What should happen?

- A. ✓ Reject the output at the validation layer and route it to a retry-with-feedback pass or human review, rather than passing it downstream as-is.
- B. Silently coerce the bad value to something plausible and continue.
- C. Pass the invalid output downstream unchanged and let a later system catch it.
- D. Disable validation going forward since it caught an inconvenient error.

Correct answer: A. A validation failure should stop bad data from silently propagating — routing it to retry-with-feedback or human review preserves data quality instead of passing a known-bad record downstream.

Q113. A classification prompt handles customer support tickets that are worded slightly differently but describe the same underlying issue inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. Assume the model will generalize correctly with zero examples provided.
- B. Add more schema fields, since the issue must be structural, not example-related.
- C. Lower the model's context window to force more consistent behavior.
- D. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

Correct answer: D. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q114. A classification prompt handles invoices from the same vendor with minor formatting differences month to month inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. Add more schema fields, since the issue must be structural, not example-related.
- B. Lower the model's context window to force more consistent behavior.
- C. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.
- D. Assume the model will generalize correctly with zero examples provided.

Correct answer: C. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q115. A classification prompt handles job postings for the same role posted on different job boards with different templates inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. Lower the model's context window to force more consistent behavior.
- B. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.
- C. Assume the model will generalize correctly with zero examples provided.
- D. Add more schema fields, since the issue must be structural, not example-related.

Correct answer: B. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q116. A classification prompt handles product reviews expressing the same sentiment with different phrasing inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.
- B. Assume the model will generalize correctly with zero examples provided.
- C. Add more schema fields, since the issue must be structural, not example-related.
- D. Lower the model's context window to force more consistent behavior.

Correct answer: A. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q117. A classification prompt handles log lines describing the same error with slightly different timestamps and IDs inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. Assume the model will generalize correctly with zero examples provided.
- B. Add more schema fields, since the issue must be structural, not example-related.
- C. Lower the model's context window to force more consistent behavior.
- D. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.

Correct answer: D. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Q118. A classification prompt handles contracts from the same template with minor clause reordering inconsistently, producing different labels for what are effectively equivalent inputs. What's the most direct fix?

- A. Add more schema fields, since the issue must be structural, not example-related.
- B. Lower the model's context window to force more consistent behavior.
- C. ✓ Add few-shot examples spanning that surface variation so the model generalizes past superficial wording differences to the underlying pattern.
- D. Assume the model will generalize correctly with zero examples provided.

Correct answer: C. Inconsistent handling of superficially-different-but-substantively-similar inputs is a strong signal that few-shot examples spanning that variation are needed, so the model learns to generalize past wording.

Tool Design & MCP Integration

Q1. Production logs show `get_customer` being called for order-related queries roughly 45% of the time it shouldn't be. Both `get_customer` and `get_order`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge `get_customer` and `get_order` into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of `get_customer` and `get_order` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q2. Production logs show `lookup_user` being called for support-ticket queries roughly 38% of the time it shouldn't be. Both `lookup_user` and `lookup_ticket`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge `lookup_user` and `lookup_ticket` into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of `lookup_user` and `lookup_ticket` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q3. Production logs show `fetch_invoice` being called for payment-related queries roughly 41% of the time it shouldn't be. Both `fetch_invoice` and `fetch_payment`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Increase the model's temperature to reduce repetitive misrouting.
- B. ✓ Expand and differentiate the descriptions of `fetch_invoice` and `fetch_payment` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- C. Build a routing classifier in front of tool selection.
- D. Merge `fetch_invoice` and `fetch_payment` into a single combined tool.

Correct answer: B. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q4. Production logs show `get_product` being called for inventory queries roughly 33% of the time it shouldn't be. Both `get_product` and `get_inventory`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. ✓ Expand and differentiate the descriptions of `get_product` and `get_inventory` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- B. Build a routing classifier in front of tool selection.
- C. Merge `get_product` and `get_inventory` into a single combined tool.
- D. Increase the model's temperature to reduce repetitive misrouting.

Correct answer: A. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q5. Production logs show search_docs being called for FAQ-type queries roughly 50% of the time it shouldn't be. Both search_docs and search_faq's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge search_docs and search_faq into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of search_docs and search_faq — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q6. Production logs show get_employee being called for department-level queries roughly 29% of the time it shouldn't be. Both get_employee and get_department's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge get_employee and get_department into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of get_employee and get_department — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q7. Production logs show fetch_shipment being called for order-status queries roughly 44% of the time it shouldn't be. Both fetch_shipment and fetch_order's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Increase the model's temperature to reduce repetitive misrouting.
- B. ✓ Expand and differentiate the descriptions of fetch_shipment and fetch_order — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- C. Build a routing classifier in front of tool selection.
- D. Merge fetch_shipment and fetch_order into a single combined tool.

Correct answer: B. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q8. Production logs show get_account being called for subscription queries roughly 36% of the time it shouldn't be. Both get_account and get_subscription's descriptions are minimal and nearly identical. What's the first fix to try?

- A. ✓ Expand and differentiate the descriptions of get_account and get_subscription — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- B. Build a routing classifier in front of tool selection.
- C. Merge get_account and get_subscription into a single combined tool.
- D. Increase the model's temperature to reduce repetitive misrouting.

Correct answer: A. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q9. Production logs show search_contacts being called for company-level queries roughly 40% of the time it shouldn't be. Both search_contacts and search_companies's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge search_contacts and search_companies into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of search_contacts and search_companies — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q10. Production logs show get_ticket being called for customer-profile queries roughly 37% of the time it shouldn't be. Both get_ticket and get_customer's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge get_ticket and get_customer into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of get_ticket and get_customer — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q11. A payment-processing tool fails with a RATE_LIMITED-type error. What should its error response look like to be useful to a coordinator?

- A. A plain string description with no structured fields at all
- B. ✓ { "errorCategory": "RATE_LIMITED", "isRetryable": true, plus any partial results }
- C. { "error": "Operation failed" }
- D. An HTTP status code with no further detail

Correct answer: B. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q12. An inventory-lookup tool fails with a NOT_FOUND-type error. What should its error response look like to be useful to a coordinator?

- A. ✓ { "errorCategory": "NOT_FOUND", "isRetryable": false, plus any partial results }
- B. { "error": "Operation failed" }
- C. An HTTP status code with no further detail
- D. A plain string description with no structured fields at all

Correct answer: A. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q13. A shipping-label tool fails with a VALIDATION_ERROR-type error. What should its error response look like to be useful to a coordinator?

- A. { "error": "Operation failed" }
- B. An HTTP status code with no further detail
- C. A plain string description with no structured fields at all
- D. ✓ { "errorCategory": "VALIDATION_ERROR", "isRetryable": true, plus any partial results }

Correct answer: D. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q14. A search tool fails with a TIMEOUT-type error. What should its error response look like to be useful to a coordinator?

- A. An HTTP status code with no further detail
- B. A plain string description with no structured fields at all
- C. ✓ { "errorCategory": "TIMEOUT", "isRetryable": true, plus any partial results }
- D. { "error": "Operation failed" }

Correct answer: C. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q15. A crm-update tool fails with a PERMISSION_DENIED-type error. What should its error response look like to be useful to a coordinator?

- A. A plain string description with no structured fields at all
- B. ✓ { "errorCategory": "PERMISSION_DENIED", "isRetryable": false, plus any partial results }
- C. { "error": "Operation failed" }
- D. An HTTP status code with no further detail

Correct answer: B. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q16. A document-generation tool fails with a UPSTREAM_UNAVAILABLE-type error. What should its error response look like to be useful to a coordinator?

- A. ✓ { "errorCategory": "UPSTREAM_UNAVAILABLE", "isRetryable": true, plus any partial results }
- B. { "error": "Operation failed" }
- C. An HTTP status code with no further detail
- D. A plain string description with no structured fields at all

Correct answer: A. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q17. A booking tool fails with a CONFLICT-type error. What should its error response look like to be useful to a coordinator?

- A. { "error": "Operation failed" }
- B. An HTTP status code with no further detail
- C. A plain string description with no structured fields at all
- D. ✓ { "errorCategory": "CONFLICT", "isRetryable": false, plus any partial results }

Correct answer: D. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q18. A file-upload tool fails with a QUOTA_EXCEEDED-type error. What should its error response look like to be useful to a coordinator?

- A. An HTTP status code with no further detail
- B. A plain string description with no structured fields at all
- C. ✓ { "errorCategory": "QUOTA_EXCEEDED", "isRetryable": true, plus any partial results }
- D. { "error": "Operation failed" }

Correct answer: C. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q19. Where should the MCP configuration for a shared internal database MCP server the whole engineering team relies on live?

- A. A hardcoded value in the tool's source code
- B. ✓ .mcp.json (project-level, version-controlled)
- C. ~/.claude.json (personal, not shared)
- D. Inside CLAUDE.md as free text

Correct answer: B. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q20. Where should the MCP configuration for a personal experimental MCP server one engineer is trying out before proposing it live?

- A. ✓ ~/.claude.json (personal, not shared)
- B. .mcp.json (project-level, version-controlled)
- C. Inside CLAUDE.md as free text
- D. A hardcoded value in the tool's source code

Correct answer: A. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q21. Where should the MCP configuration for a shared CI-status MCP server every contributor needs live?

- A. ~/.claude.json (personal, not shared)
- B. Inside CLAUDE.md as free text
- C. A hardcoded value in the tool's source code
- D. ✓ .mcp.json (project-level, version-controlled)

Correct answer: D. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q22. Where should the MCP configuration for a personal calendar MCP server unrelated to the shared project live?

- A. Inside CLAUDE.md as free text
- B. A hardcoded value in the tool's source code
- C. ✓ ~/.claude.json (personal, not shared)
- D. .mcp.json (project-level, version-controlled)

Correct answer: C. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q23. Where should the MCP configuration for a shared ticketing-system MCP server required by the whole team's workflow live?

- A. A hardcoded value in the tool's source code
- B. ✓ .mcp.json (project-level, version-controlled)
- C. ~/.claude.json (personal, not shared)
- D. Inside CLAUDE.md as free text

Correct answer: B. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q24. Where should the MCP configuration for a personal note-taking MCP server used only by one developer live?

- A. ✓ ~/.claude.json (personal, not shared)
- B. .mcp.json (project-level, version-controlled)
- C. Inside CLAUDE.md as free text
- D. A hardcoded value in the tool's source code

Correct answer: A. Shared, team-wide servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json.

Q25. Where should the MCP configuration for a shared search-index MCP server the repo depends on for every contributor live?

- A. `~/claude.json` (personal, not shared)
- B. Inside `CLAUDE.md` as free text
- C. A hardcoded value in the tool's source code
- D. ✓ `.mcp.json` (project-level, version-controlled)

Correct answer: D. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q26. Where should the MCP configuration for a personal weather MCP server with no relation to the team's actual work live?

- A. Inside `CLAUDE.md` as free text
- B. A hardcoded value in the tool's source code
- C. ✓ `~/claude.json` (personal, not shared)
- D. `.mcp.json` (project-level, version-controlled)

Correct answer: C. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q27. A team needs to supply credentials for a payments MCP server inside a version-controlled MCP config file. What's correct?

- A. Store the key in a code comment near the config.
- B. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- C. Commit the literal API key directly into the config for simplicity.
- D. Base64-encode the key and commit the encoded value.

Correct answer: B. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q28. A team needs to supply credentials for a CRM MCP server inside a version-controlled MCP config file. What's correct?

- A. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- B. Commit the literal API key directly into the config for simplicity.
- C. Base64-encode the key and commit the encoded value.
- D. Store the key in a code comment near the config.

Correct answer: A. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q29. A team needs to supply credentials for a cloud-storage MCP server inside a version-controlled MCP config file. What's correct?

- A. Commit the literal API key directly into the config for simplicity.
- B. Base64-encode the key and commit the encoded value.
- C. Store the key in a code comment near the config.
- D. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.

Correct answer: D. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q30. A team needs to supply credentials for an analytics MCP server inside a version-controlled MCP config file. What's correct?

- A. Base64-encode the key and commit the encoded value.
- B. Store the key in a code comment near the config.
- C. ✓ Use environment-variable substitution, e.g. \${API_KEY}, so no literal secret is committed to the config file.
- D. Commit the literal API key directly into the config for simplicity.

Correct answer: C. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q31. A team needs to supply credentials for an internal database MCP server inside a version-controlled MCP config file. What's correct?

- A. Store the key in a code comment near the config.
- B. ✓ Use environment-variable substitution, e.g. \${API_KEY}, so no literal secret is committed to the config file.
- C. Commit the literal API key directly into the config for simplicity.
- D. Base64-encode the key and commit the encoded value.

Correct answer: B. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q32. A team needs to supply credentials for a ticketing MCP server inside a version-controlled MCP config file. What's correct?

- A. ✓ Use environment-variable substitution, e.g. \${API_KEY}, so no literal secret is committed to the config file.
- B. Commit the literal API key directly into the config for simplicity.
- C. Base64-encode the key and commit the encoded value.
- D. Store the key in a code comment near the config.

Correct answer: A. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q33. Two tools — a tool that searches by exact ID and a tool that searches by fuzzy name match — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Give both tools identical descriptions to keep things consistent.
- B. Remove one of the two tools to eliminate the ambiguity.
- C. Rename both tools to nearly identical names for symmetry.
- D. ✓ Add concrete example queries for each, showing when an exact ID is available versus when only a name is known to each tool's description.

Correct answer: D. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q34. Two tools — a tool that reads a single record and a tool that lists many records — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Remove one of the two tools to eliminate the ambiguity.
- B. Rename both tools to nearly identical names for symmetry.
- C. ✓ Add explicit guidance on singular vs. plural use cases and expected result size to each tool's description.
- D. Give both tools identical descriptions to keep things consistent.

Correct answer: C. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q35. Two tools — a tool that creates a resource and a tool that updates an existing one — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Rename both tools to nearly identical names for symmetry.
- B. ✓ Add a clear statement of the precondition each expects (resource must not exist vs. must already exist) to each tool's description.
- C. Give both tools identical descriptions to keep things consistent.
- D. Remove one of the two tools to eliminate the ambiguity.

Correct answer: B. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q36. Two tools — a tool that queries live data and a tool that queries a cached snapshot — are being confused by the model in production. What's the highest-leverage description improvement?

- A. ✓ Add an explicit note on staleness tolerance and when live data is required to each tool's description.
- B. Give both tools identical descriptions to keep things consistent.
- C. Remove one of the two tools to eliminate the ambiguity.
- D. Rename both tools to nearly identical names for symmetry.

Correct answer: A. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q37. Two tools — a tool scoped to one region and a tool scoped globally — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Give both tools identical descriptions to keep things consistent.
- B. Remove one of the two tools to eliminate the ambiguity.
- C. Rename both tools to nearly identical names for symmetry.
- D. ✓ Add explicit input-format guidance showing which region parameter each expects to each tool's description.

Correct answer: D. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q38. Two tools — a tool for synchronous lookups and a tool for kicking off an async job — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Remove one of the two tools to eliminate the ambiguity.
- B. Rename both tools to nearly identical names for symmetry.
- C. ✓ Add explicit guidance on expected response time and when to prefer polling to each tool's description.
- D. Give both tools identical descriptions to keep things consistent.

Correct answer: C. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q39. Two tools — a tool that returns summary statistics and a tool that returns raw records — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Rename both tools to nearly identical names for symmetry.
- B. ✓ Add example queries showing when aggregate vs. granular data is actually needed to each tool's description.
- C. Give both tools identical descriptions to keep things consistent.
- D. Remove one of the two tools to eliminate the ambiguity.

Correct answer: B. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q40. Two tools — a tool for internal employees and a tool for external customers — are being confused by the model in production. What's the highest-leverage description improvement?

- A. ✓ Add an explicit statement of which caller context each tool is meant for to each tool's description.
- B. Give both tools identical descriptions to keep things consistent.
- C. Remove one of the two tools to eliminate the ambiguity.
- D. Rename both tools to nearly identical names for symmetry.

Correct answer: A. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q41. A tool design uses a single tool with a mode flag that can create, update, delete, or list resources depending on the flag's value. Is this good practice for tool design?

- A. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- B. Add more optional parameters to the same tool instead of splitting it.
- C. Leave the design as-is and rely on the system prompt to clarify usage.
- D. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.

Correct answer: D. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q42. A tool design uses a single tool that both sends emails and schedules calendar events depending on a type parameter. Is this good practice for tool design?

- A. Add more optional parameters to the same tool instead of splitting it.
- B. Leave the design as-is and rely on the system prompt to clarify usage.
- C. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- D. Keep the single multi-mode tool; fewer tools is always simpler for the model.

Correct answer: C. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q43. A tool design uses a single tool that both reads and writes configuration depending on an action parameter. Is this good practice for tool design?

- A. Leave the design as-is and rely on the system prompt to clarify usage.
- B. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- C. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- D. Add more optional parameters to the same tool instead of splitting it.

Correct answer: B. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q44. A tool design uses a single tool that handles both customer lookups and order lookups depending on a target parameter. Is this good practice for tool design?

- A. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- B. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- C. Add more optional parameters to the same tool instead of splitting it.
- D. Leave the design as-is and rely on the system prompt to clarify usage.

Correct answer: A. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q45. A tool design uses a single tool that both validates and submits a form depending on a mode flag. Is this good practice for tool design?

- A. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- B. Add more optional parameters to the same tool instead of splitting it.
- C. Leave the design as-is and rely on the system prompt to clarify usage.
- D. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.

Correct answer: D. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q46. A tool design uses a single tool that both queries and mutates inventory depending on an operation parameter. Is this good practice for tool design?

- A. Add more optional parameters to the same tool instead of splitting it.
- B. Leave the design as-is and rely on the system prompt to clarify usage.
- C. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- D. Keep the single multi-mode tool; fewer tools is always simpler for the model.

Correct answer: C. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q47. For a local development tool that only needs to run on the developer's own machine, which MCP server transport generally fits better?

- A. A local stdio server for a centrally shared, team-wide resource
- B. ✓ A local (stdio) MCP server
- C. Either transport works identically with no meaningful tradeoff
- D. A remote server for purely local, single-machine tooling

Correct answer: B. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q48. For a shared team resource that must be reachable by every contributor's session, which MCP server transport generally fits better?

- A. ✓ A remote (HTTP/SSE) MCP server
- B. Either transport works identically with no meaningful tradeoff
- C. A remote server for purely local, single-machine tooling
- D. A local stdio server for a centrally shared, team-wide resource

Correct answer: A. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q49. For a tool wrapping a local file-system operation, which MCP server transport generally fits better?

- A. Either transport works identically with no meaningful tradeoff
- B. A remote server for purely local, single-machine tooling
- C. A local stdio server for a centrally shared, team-wide resource
- D. ✓ A local (stdio) MCP server

Correct answer: D. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q50. For a tool that must be centrally maintained and updated without every developer reinstalling anything, which MCP server transport generally fits better?

- A. A remote server for purely local, single-machine tooling
- B. A local stdio server for a centrally shared, team-wide resource
- C. ✓ A remote (HTTP/SSE) MCP server
- D. Either transport works identically with no meaningful tradeoff

Correct answer: C. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q51. For a tool used only during a single developer's local debugging session, which MCP server transport generally fits better?

- A. A local stdio server for a centrally shared, team-wide resource
- B. ✓ A local (stdio) MCP server
- C. Either transport works identically with no meaningful tradeoff
- D. A remote server for purely local, single-machine tooling

Correct answer: B. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q52. For a tool that needs centralized authentication and rate-limiting shared across a whole team, which MCP server transport generally fits better?

- A. ✓ A remote (HTTP/SSE) MCP server
- B. Either transport works identically with no meaningful tradeoff
- C. A remote server for purely local, single-machine tooling
- D. A local stdio server for a centrally shared, team-wide resource

Correct answer: A. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q53. A tool's input schema has a date parameter that accepts free-form text like "next Tuesday", leading to inconsistent or wrong calls. What's the right fix?

- A. Leave the ambiguous parameter as-is and document it only in a comment.
- B. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- C. Remove the parameter entirely rather than clarifying it.
- D. ✓ Redesign the input schema to use a strict ISO-8601 date format with clear validation.

Correct answer: D. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q54. A tool's input schema has an amount parameter with no currency specified, leading to inconsistent or wrong calls. What's the right fix?

- A. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- B. Remove the parameter entirely rather than clarifying it.
- C. ✓ Redesign the input schema to use an explicit currency field alongside the amount.
- D. Leave the ambiguous parameter as-is and document it only in a comment.

Correct answer: C. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q55. A tool's input schema has an ID parameter that could be either an internal ID or an external reference number, leading to inconsistent or wrong calls. What's the right fix?

- A. Remove the parameter entirely rather than clarifying it.
- B. ✓ Redesign the input schema to use two distinct, clearly-named parameters for each ID type.
- C. Leave the ambiguous parameter as-is and document it only in a comment.
- D. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.

Correct answer: B. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q56. A tool's input schema has a free-text "query" parameter used for both search and exact lookup, leading to inconsistent or wrong calls. What's the right fix?

- A. ✓ Redesign the input schema to use separate parameters distinguishing search terms from exact identifiers.
- B. Leave the ambiguous parameter as-is and document it only in a comment.
- C. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- D. Remove the parameter entirely rather than clarifying it.

Correct answer: A. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q57. A tool's input schema has a boolean flag whose meaning isn't obvious from its name alone, leading to inconsistent or wrong calls. What's the right fix?

- A. Leave the ambiguous parameter as-is and document it only in a comment.
- B. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- C. Remove the parameter entirely rather than clarifying it.
- D. ✓ Redesign the input schema to use a clearly-named flag with an explicit description of what true/false means.

Correct answer: D. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q58. A tool's input schema has a units-less numeric parameter for a physical quantity, leading to inconsistent or wrong calls. What's the right fix?

- A. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- B. Remove the parameter entirely rather than clarifying it.
- C. ✓ Redesign the input schema to use an explicit units field or unit-suffixed parameter name.
- D. Leave the ambiguous parameter as-is and document it only in a comment.

Correct answer: C. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q59. A search tool that can return thousands of matching records in one call. How should this tool's design handle potentially very large result sets?

- A. Leave result size unbounded and rely on the model to only ask for what it needs.
- B. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- C. Let the tool always return every matching result in a single call.
- D. Cap results at a hardcoded small number with no way to retrieve more.

Correct answer: B. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q60. A log-query tool that can return an unbounded number of log lines. How should this tool's design handle potentially very large result sets?

- A. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- B. Let the tool always return every matching result in a single call.
- C. Cap results at a hardcoded small number with no way to retrieve more.
- D. Leave result size unbounded and rely on the model to only ask for what it needs.

Correct answer: A. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q61. A customer-list tool with no limit on how many customers it returns. How should this tool's design handle potentially very large result sets?

- A. Let the tool always return every matching result in a single call.
- B. Cap results at a hardcoded small number with no way to retrieve more.
- C. Leave result size unbounded and rely on the model to only ask for what it needs.
- D. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.

Correct answer: D. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q62. A transaction-history tool that could return years of records at once. How should this tool's design handle potentially very large result sets?

- A. Cap results at a hardcoded small number with no way to retrieve more.
- B. Leave result size unbounded and rely on the model to only ask for what it needs.
- C. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- D. Let the tool always return every matching result in a single call.

Correct answer: C. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q63. A document-search tool that can match a very large number of documents. How should this tool's design handle potentially very large result sets?

- A. Leave result size unbounded and rely on the model to only ask for what it needs.
- B. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- C. Let the tool always return every matching result in a single call.
- D. Cap results at a hardcoded small number with no way to retrieve more.

Correct answer: B. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q64. An inventory tool that could return every sku in the catalog at once. How should this tool's design handle potentially very large result sets?

- A. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- B. Let the tool always return every matching result in a single call.
- C. Cap results at a hardcoded small number with no way to retrieve more.
- D. Leave result size unbounded and rely on the model to only ask for what it needs.

Correct answer: A. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q65. A tool that can read and write customer records, currently sharing one broad api key across every use case. What's the better security practice?

- A. Keep one broad, shared credential across all tools for simplicity.
- B. Grant every tool full administrative access to avoid future permission issues.
- C. Rely on the system prompt to describe which permissions should be used for which tool.
- D. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.

Correct answer: D. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q66. A tool that can issue refunds, currently using the same credentials as a read-only reporting tool. What's the better security practice?

- A. Grant every tool full administrative access to avoid future permission issues.
- B. Rely on the system prompt to describe which permissions should be used for which tool.
- C. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- D. Keep one broad, shared credential across all tools for simplicity.

Correct answer: C. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q67. A tool that can delete resources, currently authenticated with the same key as a search tool. What's the better security practice?

- A. Rely on the system prompt to describe which permissions should be used for which tool.
- B. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- C. Keep one broad, shared credential across all tools for simplicity.
- D. Grant every tool full administrative access to avoid future permission issues.

Correct answer: B. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q68. A tool that can modify user permissions, sharing credentials with an unrelated notification tool. What's the better security practice?

- A. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- B. Keep one broad, shared credential across all tools for simplicity.
- C. Grant every tool full administrative access to avoid future permission issues.
- D. Rely on the system prompt to describe which permissions should be used for which tool.

Correct answer: A. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q69. A tool that can export bulk customer data, using the same broad credentials as a single-record lookup tool. What's the better security practice?

- A. Keep one broad, shared credential across all tools for simplicity.
- B. Grant every tool full administrative access to avoid future permission issues.
- C. Rely on the system prompt to describe which permissions should be used for which tool.
- D. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.

Correct answer: D. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q70. A tool that can trigger production deploys, sharing a credential with a documentation-search tool. What's the better security practice?

- A. Grant every tool full administrative access to avoid future permission issues.
- B. Rely on the system prompt to describe which permissions should be used for which tool.
- C. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- D. Keep one broad, shared credential across all tools for simplicity.

Correct answer: C. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q71. Consider a payment-charge tool that might be retried after a network timeout. Why does idempotent design matter here?

- A. Let duplicate effects happen and clean them up manually after the fact.
- B. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- C. Assume retries will never happen and design without idempotency in mind.
- D. Prevent all retries entirely, even for genuinely transient failures.

Correct answer: B. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q72. Consider an order-creation tool that could be called twice if a coordinator retries after an ambiguous failure. Why does idempotent design matter here?

- A. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- B. Assume retries will never happen and design without idempotency in mind.
- C. Prevent all retries entirely, even for genuinely transient failures.
- D. Let duplicate effects happen and clean them up manually after the fact.

Correct answer: A. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q73. Consider an email-send tool that could be triggered again by a retry after a partial failure. Why does idempotent design matter here?

- A. Assume retries will never happen and design without idempotency in mind.
- B. Prevent all retries entirely, even for genuinely transient failures.
- C. Let duplicate effects happen and clean them up manually after the fact.
- D. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.

Correct answer: D. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q74. Consider an inventory-decrement tool that might be called twice due to a retry. Why does idempotent design matter here?

- A. Prevent all retries entirely, even for genuinely transient failures.
- B. Let duplicate effects happen and clean them up manually after the fact.
- C. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- D. Assume retries will never happen and design without idempotency in mind.

Correct answer: C. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q75. Consider a resource-provisioning tool that could be invoked again after an unclear error. Why does idempotent design matter here?

- A. Let duplicate effects happen and clean them up manually after the fact.
- B. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- C. Assume retries will never happen and design without idempotency in mind.
- D. Prevent all retries entirely, even for genuinely transient failures.

Correct answer: B. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q76. Consider a notification tool that might fire twice if a retry happens after a timeout. Why does idempotent design matter here?

- A. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- B. Assume retries will never happen and design without idempotency in mind.
- C. Prevent all retries entirely, even for genuinely transient failures.
- D. Let duplicate effects happen and clean them up manually after the fact.

Correct answer: A. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q77. Before rolling out a new internal MCP server before rolling it out to the whole team, what should happen?

- A. Ship it directly to all users and monitor for complaints afterward.
- B. Test it once against a single trivial example and consider it validated.
- C. Skip testing since MCP servers are assumed to work correctly by default.
- D. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.

Correct answer: D. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q78. Before rolling out a redesigned tool description meant to fix a misrouting problem, what should happen?

- A. Test it once against a single trivial example and consider it validated.
- B. Skip testing since MCP servers are assumed to work correctly by default.
- C. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- D. Ship it directly to all users and monitor for complaints afterward.

Correct answer: C. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q79. Before rolling out a new error-response format for an existing tool, what should happen?

- A. Skip testing since MCP servers are assumed to work correctly by default.
- B. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- C. Ship it directly to all users and monitor for complaints afterward.
- D. Test it once against a single trivial example and consider it validated.

Correct answer: B. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q80. Before rolling out a newly added pagination parameter on a search tool, what should happen?

- A. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- B. Ship it directly to all users and monitor for complaints afterward.
- C. Test it once against a single trivial example and consider it validated.
- D. Skip testing since MCP servers are assumed to work correctly by default.

Correct answer: A. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q81. Before rolling out a new authentication scheme for an existing MCP server, what should happen?

- A. Ship it directly to all users and monitor for complaints afterward.
- B. Test it once against a single trivial example and consider it validated.
- C. Skip testing since MCP servers are assumed to work correctly by default.
- D. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.

Correct answer: D. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q82. Before rolling out a tool schema change that renames several fields, what should happen?

- A. Test it once against a single trivial example and consider it validated.
- B. Skip testing since MCP servers are assumed to work correctly by default.
- C. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- D. Ship it directly to all users and monitor for complaints afterward.

Correct answer: C. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q83. A widely-used lookup tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. Keep both versions forever with no deprecation signal at all.
- B. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- C. Delete the old tool immediately once a replacement exists.
- D. Silently change the old tool's behavior in place without any notice.

Correct answer: B. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q84. A payment-processing tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- B. Delete the old tool immediately once a replacement exists.
- C. Silently change the old tool's behavior in place without any notice.
- D. Keep both versions forever with no deprecation signal at all.

Correct answer: A. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q85. A search tool with many active callers needs to be replaced with an improved version. What's the safer rollout approach?

- A. Delete the old tool immediately once a replacement exists.
- B. Silently change the old tool's behavior in place without any notice.
- C. Keep both versions forever with no deprecation signal at all.
- D. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.

Correct answer: D. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q86. A reporting tool embedded in several automated workflows needs to be replaced with an improved version. What's the safer rollout approach?

- A. Silently change the old tool's behavior in place without any notice.
- B. Keep both versions forever with no deprecation signal at all.
- C. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- D. Delete the old tool immediately once a replacement exists.

Correct answer: C. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q87. A core inventory-query tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. Keep both versions forever with no deprecation signal at all.
- B. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- C. Delete the old tool immediately once a replacement exists.
- D. Silently change the old tool's behavior in place without any notice.

Correct answer: B. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q88. A shared notification tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- B. Delete the old tool immediately once a replacement exists.
- C. Silently change the old tool's behavior in place without any notice.
- D. Keep both versions forever with no deprecation signal at all.

Correct answer: A. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q89. A team is considering connecting a third-party MCP server with broad filesystem access and no published security review. What's the right posture?

- A. Connect it with full trust since it's convenient to use.
- B. Grant it broad access up front and narrow permissions later if problems appear.
- C. Assume third-party MCP servers are inherently safe because they're publicly available.
- D. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.

Correct answer: D. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q90. A team is considering connecting a community-maintained MCP server requesting access to production credentials. What's the right posture?

- A. Grant it broad access up front and narrow permissions later if problems appear.
- B. Assume third-party MCP servers are inherently safe because they're publicly available.
- C. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- D. Connect it with full trust since it's convenient to use.

Correct answer: C. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q91. A team is considering connecting an unfamiliar MCP server that requests far broader permissions than its stated purpose needs. What's the right posture?

- A. Assume third-party MCP servers are inherently safe because they're publicly available.
- B. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- C. Connect it with full trust since it's convenient to use.
- D. Grant it broad access up front and narrow permissions later if problems appear.

Correct answer: B. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q92. A team is considering connecting a third-party MCP server with no clear maintainer or update history. What's the right posture?

- A. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- B. Connect it with full trust since it's convenient to use.
- C. Grant it broad access up front and narrow permissions later if problems appear.
- D. Assume third-party MCP servers are inherently safe because they're publicly available.

Correct answer: A. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q93. A team is considering connecting an MCP server whose source code isn't available for review before granting it access. What's the right posture?

- A. Connect it with full trust since it's convenient to use.
- B. Grant it broad access up front and narrow permissions later if problems appear.
- C. Assume third-party MCP servers are inherently safe because they're publicly available.
- D. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.

Correct answer: D. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q94. A team is considering connecting a third-party server requesting access to customer PII with no stated data-handling policy. What's the right posture?

- A. Grant it broad access up front and narrow permissions later if problems appear.
- B. Assume third-party MCP servers are inherently safe because they're publicly available.
- C. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- D. Connect it with full trust since it's convenient to use.

Correct answer: C. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q95. A payment tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only that the tool was called, with no arguments or result detail.
- B. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- C. Log only failures, since successful calls don't need a record.
- D. Skip logging to keep the tool's implementation simpler.

Correct answer: B. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q96. A data-export tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- B. Log only failures, since successful calls don't need a record.
- C. Skip logging to keep the tool's implementation simpler.
- D. Log only that the tool was called, with no arguments or result detail.

Correct answer: A. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q97. A permission-change tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only failures, since successful calls don't need a record.
- B. Skip logging to keep the tool's implementation simpler.
- C. Log only that the tool was called, with no arguments or result detail.
- D. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.

Correct answer: D. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q98. A customer-communication tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Skip logging to keep the tool's implementation simpler.
- B. Log only that the tool was called, with no arguments or result detail.
- C. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- D. Log only failures, since successful calls don't need a record.

Correct answer: C. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q99. A deployment tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only that the tool was called, with no arguments or result detail.
- B. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- C. Log only failures, since successful calls don't need a record.
- D. Skip logging to keep the tool's implementation simpler.

Correct answer: B. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q100. A bulk-delete tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- B. Log only failures, since successful calls don't need a record.
- C. Skip logging to keep the tool's implementation simpler.
- D. Log only that the tool was called, with no arguments or result detail.

Correct answer: A. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q101. Which MCP primitive best fits exposing a static reference document the model can read as needed?

- A. An MCP tool
- B. An MCP prompt
- C. A CLAUDE.md entry
- D. ✓ An MCP resource

Correct answer: D. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q102. Which MCP primitive best fits exposing an action the model can invoke to change or fetch live state?

- A. An MCP prompt
- B. A CLAUDE.md entry
- C. ✓ An MCP tool
- D. An MCP resource

Correct answer: C. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q103. Which MCP primitive best fits exposing a reusable prompt template other clients can select and fill in?

- A. A CLAUDE.md entry
- B. ✓ An MCP prompt
- C. An MCP resource
- D. An MCP tool

Correct answer: B. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q104. Which MCP primitive best fits exposing a read-only configuration file for the model to consult?

- A. ✓ An MCP resource
- B. An MCP tool
- C. An MCP prompt
- D. A CLAUDE.md entry

Correct answer: A. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q105. Which MCP primitive best fits exposing an operation that sends an email on the model's behalf?

- A. An MCP resource
- B. An MCP prompt
- C. A CLAUDE.md entry
- D. ✓ An MCP tool

Correct answer: D. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q106. Which MCP primitive best fits exposing a curated, reusable instruction template for a common task?

- A. An MCP tool
- B. A CLAUDE.md entry
- C. ✓ An MCP prompt
- D. An MCP resource

Correct answer: C. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q107. A tool wraps a third-party search API with a strict per-minute call limit. How should the tool handle hitting the rate limit?

- A. Disable the tool entirely rather than handling the rate limit gracefully.
- B. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- C. Let calls fail silently with no indication that a rate limit was hit.
- D. Retry immediately and repeatedly with no backoff until it succeeds.

Correct answer: B. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q108. A tool wraps an internal service that degrades under bursty tool-call traffic. How should the tool handle hitting the rate limit?

- A. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- B. Let calls fail silently with no indication that a rate limit was hit.
- C. Retry immediately and repeatedly with no backoff until it succeeds.
- D. Disable the tool entirely rather than handling the rate limit gracefully.

Correct answer: A. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q109. A tool wraps a payment gateway with published rate limits. How should the tool handle hitting the rate limit?

- A. Let calls fail silently with no indication that a rate limit was hit.
- B. Retry immediately and repeatedly with no backoff until it succeeds.
- C. Disable the tool entirely rather than handling the rate limit gracefully.
- D. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.

Correct answer: D. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q110. A tool wraps a shared internal database that slows down under concurrent load from many agent sessions. How should the tool handle hitting the rate limit?

- A. Retry immediately and repeatedly with no backoff until it succeeds.
- B. Disable the tool entirely rather than handling the rate limit gracefully.
- C. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- D. Let calls fail silently with no indication that a rate limit was hit.

Correct answer: C. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q111. A tool wraps an external weather API with a modest free-tier call quota. How should the tool handle hitting the rate limit?

- A. Disable the tool entirely rather than handling the rate limit gracefully.
- B. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- C. Let calls fail silently with no indication that a rate limit was hit.
- D. Retry immediately and repeatedly with no backoff until it succeeds.

Correct answer: B. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q112. A tool wraps a shared logging service that throttles high-volume callers. How should the tool handle hitting the rate limit?

- A. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- B. Let calls fail silently with no indication that a rate limit was hit.
- C. Retry immediately and repeatedly with no backoff until it succeeds.
- D. Disable the tool entirely rather than handling the rate limit gracefully.

Correct answer: A. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q113. A workflow composes a lookup tool followed by a calculation tool followed by a notification tool. What matters most for this chain to be reliable in production?

- A. Assume the chain will always succeed end-to-end with no failure handling.
- B. Combine all steps into one large tool to avoid composition issues.
- C. Let each tool run independently with no attention to how its output feeds the next step.
- D. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.

Correct answer: D. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q114. A workflow composes a validation tool followed by a submission tool. What matters most for this chain to be reliable in production?

- A. Combine all steps into one large tool to avoid composition issues.
- B. Let each tool run independently with no attention to how its output feeds the next step.
- C. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- D. Assume the chain will always succeed end-to-end with no failure handling.

Correct answer: C. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q115. A workflow composes a search tool followed by a summarization tool followed by an export tool. What matters most for this chain to be reliable in production?

- A. Let each tool run independently with no attention to how its output feeds the next step.
- B. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- C. Assume the chain will always succeed end-to-end with no failure handling.
- D. Combine all steps into one large tool to avoid composition issues.

Correct answer: B. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q116. A workflow composes an authentication tool followed by a data-fetch tool that requires its token. What matters most for this chain to be reliable in production?

- A. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- B. Assume the chain will always succeed end-to-end with no failure handling.
- C. Combine all steps into one large tool to avoid composition issues.
- D. Let each tool run independently with no attention to how its output feeds the next step.

Correct answer: A. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q117. A workflow composes a data-cleaning tool followed by an analysis tool. What matters most for this chain to be reliable in production?

- A. Assume the chain will always succeed end-to-end with no failure handling.
- B. Combine all steps into one large tool to avoid composition issues.
- C. Let each tool run independently with no attention to how its output feeds the next step.
- D. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.

Correct answer: D. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q118. A workflow composes a draft-generation tool followed by a review tool followed by a publish tool. What matters most for this chain to be reliable in production?

- A. Combine all steps into one large tool to avoid composition issues.
- B. Let each tool run independently with no attention to how its output feeds the next step.
- C. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- D. Assume the chain will always succeed end-to-end with no failure handling.

Correct answer: C. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Context Management & Reliability

Q1. Over a long session, a refund amount of \$482.17 gets progressively summarized until it appears only as "a refund amount". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q2. Over a long session, an order ID of #A-88214 gets progressively summarized until it appears only as "an order". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q3. Over a long session, a meeting scheduled for 2:15 PM on March 14th gets progressively summarized until it appears only as "a meeting sometime soon". What's the correct fix?

- A. Accept the vague version since narrative continuity matters more than precision.
- B. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- C. Disable summarization for the entire session to avoid any loss.
- D. Ask the model to try harder to remember precise values during summarization.

Correct answer: B. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q4. Over a long session, a contract effective date of June 1, 2027 gets progressively summarized until it appears only as "an upcoming effective date". What's the correct fix?

- A. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- B. Disable summarization for the entire session to avoid any loss.
- C. Ask the model to try harder to remember precise values during summarization.
- D. Accept the vague version since narrative continuity matters more than precision.

Correct answer: A. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q5. Over a long session, a customer account number ending in 4471 gets progressively summarized until it appears only as "the customer's account". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q6. Over a long session, a shipment tracking number gets progressively summarized until it appears only as "a shipment". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q7. Over a long session, a specific discount of 17% gets progressively summarized until it appears only as "a discount". What's the correct fix?

- A. Accept the vague version since narrative continuity matters more than precision.
- B. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- C. Disable summarization for the entire session to avoid any loss.
- D. Ask the model to try harder to remember precise values during summarization.

Correct answer: B. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q8. Over a long session, an exact patient dosage of 250mg gets progressively summarized until it appears only as "a dosage". What's the correct fix?

- A. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- B. Disable summarization for the entire session to avoid any loss.
- C. Ask the model to try harder to remember precise values during summarization.
- D. Accept the vague version since narrative continuity matters more than precision.

Correct answer: A. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q9. Over a long session, a precise flight number and gate gets progressively summarized until it appears only as "a flight". What's the correct fix?

- A. Disable summarization for the entire session to avoid any loss.
- B. Ask the model to try harder to remember precise values during summarization.
- C. Accept the vague version since narrative continuity matters more than precision.
- D. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.

Correct answer: D. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q10. Over a long session, an exact invoice total of \$3,204.56 gets progressively summarized until it appears only as "an invoice amount". What's the correct fix?

- A. Ask the model to try harder to remember precise values during summarization.
- B. Accept the vague version since narrative continuity matters more than precision.
- C. ✓ Pull the transactional fact into a structured block that lives outside the summarization pipeline and is included verbatim in every prompt.
- D. Disable summarization for the entire session to avoid any loss.

Correct answer: C. Structured, unsummarized transactional facts prevent precision loss regardless of how aggressively the surrounding narrative gets compressed — disabling summarization entirely isn't necessary or scalable.

Q11. A 75,000-token aggregated research document has its single most important finding located around token 40,000, with no heading structure. What's the risk, and what's the fix?

- A. Shorten the whole document uniformly with no regard for where the key finding sits.
- B. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- C. Leave the document structure unchanged; length alone isn't a reliability concern.
- D. Move the finding to be repeated only at the very end of the document.

Correct answer: B. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q12. A 60,000-token legal-review compilation has its single most important finding located around the middle third of the document, with no heading structure. What's the risk, and what's the fix?

- A. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- B. Leave the document structure unchanged; length alone isn't a reliability concern.
- C. Move the finding to be repeated only at the very end of the document.
- D. Shorten the whole document uniformly with no regard for where the key finding sits.

Correct answer: A. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q13. A 90,000-token multi-source competitive analysis has its single most important finding located around roughly the halfway point, with no heading structure. What's the risk, and what's the fix?

- A. Leave the document structure unchanged; length alone isn't a reliability concern.
- B. Move the finding to be repeated only at the very end of the document.
- C. Shorten the whole document uniformly with no regard for where the key finding sits.
- D. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.

Correct answer: D. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q14. A 50,000-token customer-feedback aggregation has its single most important finding located around a section buried well past the introduction, with no heading structure. What's the risk, and what's the fix?

- A. Move the finding to be repeated only at the very end of the document.
- B. Shorten the whole document uniformly with no regard for where the key finding sits.
- C. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- D. Leave the document structure unchanged; length alone isn't a reliability concern.

Correct answer: C. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q15. A 100,000-token compiled audit report has its single most important finding located around a finding deep in the middle section, with no heading structure. What's the risk, and what's the fix?

- A. Shorten the whole document uniformly with no regard for where the key finding sits.
- B. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- C. Leave the document structure unchanged; length alone isn't a reliability concern.
- D. Move the finding to be repeated only at the very end of the document.

Correct answer: B. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q16. A 45,000-token merged incident-review document has its single most important finding located around a root-cause note buried mid-document, with no heading structure. What's the risk, and what's the fix?

- A. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- B. Leave the document structure unchanged; length alone isn't a reliability concern.
- C. Move the finding to be repeated only at the very end of the document.
- D. Shorten the whole document uniformly with no regard for where the key finding sits.

Correct answer: A. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q17. An 80,000-token combined market-research report has its single most important finding located around a critical caveat placed mid-document, with no heading structure. What's the risk, and what's the fix?

- A. Leave the document structure unchanged; length alone isn't a reliability concern.
- B. Move the finding to be repeated only at the very end of the document.
- C. Shorten the whole document uniformly with no regard for where the key finding sits.
- D. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.

Correct answer: D. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q18. A 65,000-token merged due-diligence file has its single most important finding located around a key risk factor buried in the middle, with no heading structure. What's the risk, and what's the fix?

- A. Move the finding to be repeated only at the very end of the document.
- B. Shorten the whole document uniformly with no regard for where the key finding sits.
- C. ✓ Put the key finding in a summary at the top of the document and use explicit section headings throughout.
- D. Leave the document structure unchanged; length alone isn't a reliability concern.

Correct answer: C. Long-context models attend more reliably to the start and end of an input; content buried in the middle with no navigation anchors is at real risk of being under-weighted — front-loading key findings and adding section headings mitigates this.

Q19. A lookup_order call returns 40 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Increase the context window instead of trimming the tool output.
- B. ✓ Use a PostToolUse hook to trim the result down to the 5 relevant fields before it enters context.
- C. Pass all fields through unchanged and let the model ignore what it doesn't need.
- D. Call the tool twice and merge the results for redundancy.

Correct answer: B. Trimming at the tool-output layer via a PostToolUse hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q20. A `get_customer_profile` call returns 35 fields when only 6 are relevant to the current task. What's the best way to reduce the context footprint?

- A. ✓ Use a `PostToolUse` hook to trim the result down to the 6 relevant fields before it enters context.
- B. Pass all fields through unchanged and let the model ignore what it doesn't need.
- C. Call the tool twice and merge the results for redundancy.
- D. Increase the context window instead of trimming the tool output.

Correct answer: A. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q21. A `fetch_shipment_details` call returns 28 fields when only 4 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Pass all fields through unchanged and let the model ignore what it doesn't need.
- B. Call the tool twice and merge the results for redundancy.
- C. Increase the context window instead of trimming the tool output.
- D. ✓ Use a `PostToolUse` hook to trim the result down to the 4 relevant fields before it enters context.

Correct answer: D. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q22. A `get_employee_record` call returns 32 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Call the tool twice and merge the results for redundancy.
- B. Increase the context window instead of trimming the tool output.
- C. ✓ Use a `PostToolUse` hook to trim the result down to the 5 relevant fields before it enters context.
- D. Pass all fields through unchanged and let the model ignore what it doesn't need.

Correct answer: C. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q23. A `fetch_transaction` call returns 25 fields when only 4 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Increase the context window instead of trimming the tool output.
- B. ✓ Use a `PostToolUse` hook to trim the result down to the 4 relevant fields before it enters context.
- C. Pass all fields through unchanged and let the model ignore what it doesn't need.
- D. Call the tool twice and merge the results for redundancy.

Correct answer: B. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q24. A `get_property_listing` call returns 30 fields when only 6 are relevant to the current task. What's the best way to reduce the context footprint?

- A. ✓ Use a `PostToolUse` hook to trim the result down to the 6 relevant fields before it enters context.
- B. Pass all fields through unchanged and let the model ignore what it doesn't need.
- C. Call the tool twice and merge the results for redundancy.
- D. Increase the context window instead of trimming the tool output.

Correct answer: A. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q25. A `fetch_patient_record` call returns 45 fields when only 7 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Pass all fields through unchanged and let the model ignore what it doesn't need.
- B. Call the tool twice and merge the results for redundancy.
- C. Increase the context window instead of trimming the tool output.
- D. ✓ Use a `PostToolUse` hook to trim the result down to the 7 relevant fields before it enters context.

Correct answer: D. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q26. A `get_vendor_profile` call returns 27 fields when only 5 are relevant to the current task. What's the best way to reduce the context footprint?

- A. Call the tool twice and merge the results for redundancy.
- B. Increase the context window instead of trimming the tool output.
- C. ✓ Use a `PostToolUse` hook to trim the result down to the 5 relevant fields before it enters context.
- D. Pass all fields through unchanged and let the model ignore what it doesn't need.

Correct answer: C. Trimming at the tool-output layer via a `PostToolUse` hook removes noise before it ever consumes context, which is more reliable than hoping the model filters irrelevant fields itself.

Q27. Is "the user explicitly asks to speak with a human" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Reliable — this is explicit or structurally verifiable.
- C. Unreliable — explicit requests should never be trusted at face value.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q28. Is "a sentiment classifier flags the conversation as negative" a reliable trigger for escalating to a human?

- A. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- B. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q29. Is "the agent has made several genuine attempts and cannot proceed further" a reliable trigger for escalating to a human?

- A. Unreliable — explicit requests should never be trusted at face value.
- B. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- C. This trigger type is not something a production agent should ever consider.
- D. ✓ Reliable — this is explicit or structurally verifiable.

Correct answer: D. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q30. Is "the model reports low confidence in its own answer" a reliable trigger for escalating to a human?

- A. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- B. This trigger type is not something a production agent should ever consider.
- C. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- D. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.

Correct answer: C. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q31. Is "the request falls into a genuine policy gap the agent has no authority to interpret" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Reliable — this is explicit or structurally verifiable.
- C. Unreliable — explicit requests should never be trusted at face value.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q32. Is "an automatic soft-label classifier scores the query as "complex"" a reliable trigger for escalating to a human?

- A. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- B. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q33. Is "the user's tone seems frustrated based on word choice alone" a reliable trigger for escalating to a human?

- A. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- B. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- C. This trigger type is not something a production agent should ever consider.
- D. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.

Correct answer: D. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q34. Is "the agent has hit a hard, explicit constraint it cannot satisfy through any available action" a reliable trigger for escalating to a human?

- A. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- B. This trigger type is not something a production agent should ever consider.
- C. ✓ Reliable — this is explicit or structurally verifiable.
- D. Unreliable — explicit requests should never be trusted at face value.

Correct answer: C. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q35. Is "a keyword-based heuristic flags certain words as "sensitive"" a reliable trigger for escalating to a human?

- A. This trigger type is not something a production agent should ever consider.
- B. ✓ Unreliable — this is a soft, internally-generated or statistical signal that can be miscalibrated.
- C. Reliable — statistical or self-reported signals are just as trustworthy as explicit requests.
- D. Reliability depends only on how the trigger is phrased in code, not its underlying nature.

Correct answer: B. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q36. Is "the user directly states the automated response isn't resolving their issue and asks for a person" a reliable trigger for escalating to a human?

- A. ✓ Reliable — this is explicit or structurally verifiable.
- B. Unreliable — explicit requests should never be trusted at face value.
- C. Reliability depends only on how the trigger is phrased in code, not its underlying nature.
- D. This trigger type is not something a production agent should ever consider.

Correct answer: A. Reliable triggers are explicit (a stated request) or structurally verifiable (a genuine policy gap, demonstrated inability to proceed); soft signals like sentiment, self-reported confidence, or keyword heuristics are prone to miscalibration and are the exam's classic distractors.

Q37. A long-running support session needs to reliably track a customer's loyalty-tier status and points balance across many turns. What's the most reliable way to maintain this?

- A. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- B. Ask the model to recall it from memory whenever it's needed.
- C. Store it once at the start and never re-check it as the session continues.
- D. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.

Correct answer: D. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q38. A long-running support session needs to reliably track a patient's known allergies and current medications across many turns. What's the most reliable way to maintain this?

- A. Ask the model to recall it from memory whenever it's needed.
- B. Store it once at the start and never re-check it as the session continues.
- C. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- D. Let it live only inside the narrative conversation history, to be re-derived from context each time.

Correct answer: C. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q39. A long-running support session needs to reliably track a case's SLA deadline and priority level across many turns. What's the most reliable way to maintain this?

- A. Store it once at the start and never re-check it as the session continues.
- B. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- C. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- D. Ask the model to recall it from memory whenever it's needed.

Correct answer: B. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q40. A long-running support session needs to reliably track an order's shipping address and payment method across many turns. What's the most reliable way to maintain this?

- A. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- B. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- C. Ask the model to recall it from memory whenever it's needed.
- D. Store it once at the start and never re-check it as the session continues.

Correct answer: A. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q41. A long-running support session needs to reliably track a project's budget cap and approved scope across many turns. What's the most reliable way to maintain this?

- A. Let it live only inside the narrative conversation history, to be re-derived from context each time.
- B. Ask the model to recall it from memory whenever it's needed.
- C. Store it once at the start and never re-check it as the session continues.
- D. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.

Correct answer: D. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q42. A long-running support session needs to reliably track a contract's renewal date and cancellation terms across many turns. What's the most reliable way to maintain this?

- A. Ask the model to recall it from memory whenever it's needed.
- B. Store it once at the start and never re-check it as the session continues.
- C. ✓ Maintain it in a structured fact block, included verbatim in every prompt, separate from the narrative conversation summary.
- D. Let it live only inside the narrative conversation history, to be re-derived from context each time.

Correct answer: C. Facts that must stay precise and available at any point are best kept in a structured block outside the summarized narrative — narrative summaries are the part of context most prone to lossy compression.

Q43. In a production agent system, a primary data source becomes temporarily unavailable. What's the reliability-conscious way to handle this?

- A. Retry indefinitely with no fallback or escalation path.
- B. ✓ Fall back to a cached or secondary source and clearly flag that the answer may be based on slightly stale data.
- C. Fail the entire task outright with no partial value delivered.
- D. Proceed as if nothing went wrong, with no indication anything was degraded.

Correct answer: B. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q44. In a production agent system, a specialized tool call fails but a more general tool could partially answer the question. What's the reliability-conscious way to handle this?

- A. ✓ Use the general tool as a fallback and note the reduced precision rather than failing outright.
- B. Fail the entire task outright with no partial value delivered.
- C. Proceed as if nothing went wrong, with no indication anything was degraded.
- D. Retry indefinitely with no fallback or escalation path.

Correct answer: A. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q45. In a production agent system, a subagent times out but partial results were already gathered. What's the reliability-conscious way to handle this?

- A. Fail the entire task outright with no partial value delivered.
- B. Proceed as if nothing went wrong, with no indication anything was degraded.
- C. Retry indefinitely with no fallback or escalation path.
- D. ✓ Return the partial results with a clear note about what's missing, rather than discarding everything.

Correct answer: D. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q46. In a production agent system, the primary model is temporarily rate-limited. What's the reliability-conscious way to handle this?

- A. Proceed as if nothing went wrong, with no indication anything was degraded.
- B. Retry indefinitely with no fallback or escalation path.
- C. ✓ Queue or gracefully delay the request with a clear status, rather than silently dropping it.
- D. Fail the entire task outright with no partial value delivered.

Correct answer: C. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q47. In a production agent system, an external API used for enrichment is down. What's the reliability-conscious way to handle this?

- A. Retry indefinitely with no fallback or escalation path.
- B. ✓ Proceed with the core task using only the data that's actually available, flagging the missing enrichment.
- C. Fail the entire task outright with no partial value delivered.
- D. Proceed as if nothing went wrong, with no indication anything was degraded.

Correct answer: B. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q48. In a production agent system, a required field can't be retrieved from any available source. What's the reliability-conscious way to handle this?

- A. ✓ Explicitly mark the field as unavailable rather than fabricating a plausible-looking value.
- B. Fail the entire task outright with no partial value delivered.
- C. Proceed as if nothing went wrong, with no indication anything was degraded.
- D. Retry indefinitely with no fallback or escalation path.

Correct answer: A. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q49. In a production agent system, a downstream service needed for final formatting is unavailable. What's the reliability-conscious way to handle this?

- A. Fail the entire task outright with no partial value delivered.
- B. Proceed as if nothing went wrong, with no indication anything was degraded.
- C. Retry indefinitely with no fallback or escalation path.
- D. ✓ Return the unformatted result with a note, rather than blocking the entire task on formatting.

Correct answer: D. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q50. In a production agent system, a validation service is temporarily down. What's the reliability-conscious way to handle this?

- A. Proceed as if nothing went wrong, with no indication anything was degraded.
- B. Retry indefinitely with no fallback or escalation path.
- C. ✓ Hold the result for validation once the service returns, rather than skipping validation silently.
- D. Fail the entire task outright with no partial value delivered.

Correct answer: C. Graceful degradation preserves as much value as possible while being transparent about what's missing or uncertain — the failure modes to avoid are silent full failure and silent false confidence.

Q51. For a customer-support session that can run for hours across many back-and-forth turns, how should context be managed as the session grows very long?

- A. Keep every single turn verbatim forever regardless of context-window limits.
- B. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- C. Let the context window fill up naturally and let truncation happen wherever it falls.
- D. Reset the session from scratch periodically with no continuity at all.

Correct answer: B. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q52. For a long research task that accumulates many tool results over time, how should context be managed as the session grows very long?

- A. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- B. Let the context window fill up naturally and let truncation happen wherever it falls.
- C. Reset the session from scratch periodically with no continuity at all.
- D. Keep every single turn verbatim forever regardless of context-window limits.

Correct answer: A. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q53. For a multi-day project-tracking assistant with a large accumulated history, how should context be managed as the session grows very long?

- A. Let the context window fill up naturally and let truncation happen wherever it falls.
- B. Reset the session from scratch periodically with no continuity at all.
- C. Keep every single turn verbatim forever regardless of context-window limits.
- D. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.

Correct answer: D. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q54. For an extended code-review session touching many files over time, how should context be managed as the session grows very long?

- A. Reset the session from scratch periodically with no continuity at all.
- B. Keep every single turn verbatim forever regardless of context-window limits.
- C. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- D. Let the context window fill up naturally and let truncation happen wherever it falls.

Correct answer: C. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q55. For a long-running incident-response session with a growing timeline of events, how should context be managed as the session grows very long?

- A. Keep every single turn verbatim forever regardless of context-window limits.
- B. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- C. Let the context window fill up naturally and let truncation happen wherever it falls.
- D. Reset the session from scratch periodically with no continuity at all.

Correct answer: B. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q56. For a multi-session onboarding assistant that should recall earlier sessions, how should context be managed as the session grows very long?

- A. ✓ Compact older, less-relevant history while explicitly preserving structured facts and recent context, rather than letting the window fill until truncation happens uncontrolled.
- B. Let the context window fill up naturally and let truncation happen wherever it falls.
- C. Reset the session from scratch periodically with no continuity at all.
- D. Keep every single turn verbatim forever regardless of context-window limits.

Correct answer: A. Deliberate, structured compaction — preserving key facts and recent context while summarizing or dropping less-relevant older material — is more reliable than either uncontrolled truncation or an ever-growing unmanaged window.

Q57. For a customer-support agent that has been running for an unusually long session, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Assume long sessions are always as reliable as short ones with no extra monitoring.
- B. Only check for drift if the user explicitly complains.
- C. Restart the session automatically at a fixed time interval with no regard for actual state.
- D. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.

Correct answer: D. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q58. For a research agent that has gone through many rounds of tool calls, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Only check for drift if the user explicitly complains.
- B. Restart the session automatically at a fixed time interval with no regard for actual state.
- C. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- D. Assume long sessions are always as reliable as short ones with no extra monitoring.

Correct answer: C. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q59. For a long-running data-cleanup agent operating over many hours, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Restart the session automatically at a fixed time interval with no regard for actual state.
- B. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- C. Assume long sessions are always as reliable as short ones with no extra monitoring.
- D. Only check for drift if the user explicitly complains.

Correct answer: B. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q60. For an extended code-migration agent working across a large codebase, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- B. Assume long sessions are always as reliable as short ones with no extra monitoring.
- C. Only check for drift if the user explicitly complains.
- D. Restart the session automatically at a fixed time interval with no regard for actual state.

Correct answer: A. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q61. For a multi-day monitoring agent that continuously ingests new data, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Assume long sessions are always as reliable as short ones with no extra monitoring.
- B. Only check for drift if the user explicitly complains.
- C. Restart the session automatically at a fixed time interval with no regard for actual state.
- D. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.

Correct answer: D. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q62. For a long conversational assistant session spanning many topic changes, what reliability practice helps catch the session gradually losing track of its original goal or constraints?

- A. Only check for drift if the user explicitly complains.
- B. Restart the session automatically at a fixed time interval with no regard for actual state.
- C. ✓ Monitor for drift — track whether the agent's stated goal, key facts, and constraints still match what was true at the start, and flag or reset when they've diverged.
- D. Assume long sessions are always as reliable as short ones with no extra monitoring.

Correct answer: C. Long sessions are prone to gradual drift as context gets summarized and reshaped; explicit monitoring against the original goal and constraints catches this before it silently degrades output quality.

Q63. A financial-approval agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only failures, since successful runs don't need a trace.
- B. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- C. Log only the final output, with no record of how the agent got there.
- D. Keep no logs at all to minimize storage and complexity.

Correct answer: B. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q64. A medical-information assistant operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- B. Log only the final output, with no record of how the agent got there.
- C. Keep no logs at all to minimize storage and complexity.
- D. Log only failures, since successful runs don't need a trace.

Correct answer: A. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q65. A legal-document-review agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only the final output, with no record of how the agent got there.
- B. Keep no logs at all to minimize storage and complexity.
- C. Log only failures, since successful runs don't need a trace.
- D. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.

Correct answer: D. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q66. A customer-data-access agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Keep no logs at all to minimize storage and complexity.
- B. Log only failures, since successful runs don't need a trace.
- C. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- D. Log only the final output, with no record of how the agent got there.

Correct answer: C. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q67. An hr-decision-support agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. Log only failures, since successful runs don't need a trace.
- B. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- C. Log only the final output, with no record of how the agent got there.
- D. Keep no logs at all to minimize storage and complexity.

Correct answer: B. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q68. A compliance-monitoring agent operates in a high-stakes domain where decisions may later need to be reviewed. What logging practice supports reliability and accountability here?

- A. ✓ Maintain a full audit trail of inputs, tool calls, intermediate reasoning where feasible, and outputs, tied to a session/trace ID.
- B. Log only the final output, with no record of how the agent got there.
- C. Keep no logs at all to minimize storage and complexity.
- D. Log only failures, since successful runs don't need a trace.

Correct answer: A. In high-stakes domains, a full, traceable audit record — not just the final answer — is what makes it possible to review, debug, and be accountable for how a system reached a given output.

Q69. A reliability-conscious agent system includes a payment-confirmation step that could be retried after a network blip that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Avoid retries altogether, even for clearly transient failures.
- B. Let the duplicate effect happen and reconcile it with a manual process afterward.
- C. Retry with a brand-new identifier each time to be safe.
- D. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.

Correct answer: D. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q70. A reliability-conscious agent system includes an order-submission step that might be retried after an ambiguous timeout that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Let the duplicate effect happen and reconcile it with a manual process afterward.
- B. Retry with a brand-new identifier each time to be safe.
- C. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- D. Avoid retries altogether, even for clearly transient failures.

Correct answer: C. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q71. A reliability-conscious agent system includes a notification-send step that could fire twice after a retry that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Retry with a brand-new identifier each time to be safe.
- B. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- C. Avoid retries altogether, even for clearly transient failures.
- D. Let the duplicate effect happen and reconcile it with a manual process afterward.

Correct answer: B. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q72. A reliability-conscious agent system includes a resource-provisioning step that might be invoked twice due to a retry that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- B. Avoid retries altogether, even for clearly transient failures.
- C. Let the duplicate effect happen and reconcile it with a manual process afterward.
- D. Retry with a brand-new identifier each time to be safe.

Correct answer: A. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q73. A reliability-conscious agent system includes an inventory-adjustment step that could be retried after a partial failure that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Avoid retries altogether, even for clearly transient failures.
- B. Let the duplicate effect happen and reconcile it with a manual process afterward.
- C. Retry with a brand-new identifier each time to be safe.
- D. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.

Correct answer: D. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q74. A reliability-conscious agent system includes a record-creation step that might be retried after an unclear response that may need to be retried after a transient failure. How should this be designed to stay safe under retry?

- A. Let the duplicate effect happen and reconcile it with a manual process afterward.
- B. Retry with a brand-new identifier each time to be safe.
- C. ✓ Use an idempotency key so a retried call is recognized as the same logical operation and doesn't duplicate its effect.
- D. Avoid retries altogether, even for clearly transient failures.

Correct answer: C. Idempotency keys let a retried call be recognized as a repeat of the same operation, preventing a legitimate retry from silently duplicating a real-world effect.

Q75. A downstream enrichment api that has started failing repeatedly. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- B. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- C. Keep retrying every call immediately with no pause, regardless of the failure rate.
- D. Wait indefinitely for the dependency to recover with no fallback in the meantime.

Correct answer: B. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q76. A subagent type that has failed on the last several consecutive attempts. What reliability pattern helps prevent this from cascading into a larger outage?

- A. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- B. Keep retrying every call immediately with no pause, regardless of the failure rate.
- C. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- D. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.

Correct answer: A. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q77. An external search provider returning errors on most recent calls. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Keep retrying every call immediately with no pause, regardless of the failure rate.
- B. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- C. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- D. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.

Correct answer: D. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q78. A database that has been timing out on the majority of recent queries. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- B. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- C. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- D. Keep retrying every call immediately with no pause, regardless of the failure rate.

Correct answer: C. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q79. A third-party notification service failing on nearly every recent call. What reliability pattern helps prevent this from cascading into a larger outage?

- A. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.
- B. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- C. Keep retrying every call immediately with no pause, regardless of the failure rate.
- D. Wait indefinitely for the dependency to recover with no fallback in the meantime.

Correct answer: B. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q80. An internal microservice returning errors far more often than usual. What reliability pattern helps prevent this from cascading into a larger outage?

- A. ✓ Trip a circuit breaker: stop sending calls to the failing dependency for a cooldown period and fail fast (or fall back) instead of continuing to hammer it.
- B. Keep retrying every call immediately with no pause, regardless of the failure rate.
- C. Wait indefinitely for the dependency to recover with no fallback in the meantime.
- D. Silently ignore the repeated failures and keep proceeding as if calls were succeeding.

Correct answer: A. A circuit breaker stops repeatedly hammering a dependency that's clearly failing, both protecting the dependency and letting the caller fail fast or fall back instead of hanging or degrading everything downstream.

Q81. A research agent that issues many rapid tool calls in a short window starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Retry every rate-limited call immediately in a tight loop until it succeeds.
- B. Ignore rate-limit responses and keep sending calls at the same pace.
- C. Abandon the task entirely the first time a rate limit is hit.
- D. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.

Correct answer: D. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q82. A bulk-processing agent iterating over thousands of records starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Ignore rate-limit responses and keep sending calls at the same pace.
- B. Abandon the task entirely the first time a rate limit is hit.
- C. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- D. Retry every rate-limited call immediately in a tight loop until it succeeds.

Correct answer: C. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q83. A multi-subagent system where several subagents call the same api concurrently starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Abandon the task entirely the first time a rate limit is hit.
- B. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- C. Retry every rate-limited call immediately in a tight loop until it succeeds.
- D. Ignore rate-limit responses and keep sending calls at the same pace.

Correct answer: B. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q84. A monitoring agent that polls an external service frequently starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- B. Retry every rate-limited call immediately in a tight loop until it succeeds.
- C. Ignore rate-limit responses and keep sending calls at the same pace.
- D. Abandon the task entirely the first time a rate limit is hit.

Correct answer: A. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q85. A data-enrichment pipeline calling a rate-limited third-party api starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Retry every rate-limited call immediately in a tight loop until it succeeds.
- B. Ignore rate-limit responses and keep sending calls at the same pace.
- C. Abandon the task entirely the first time a rate limit is hit.
- D. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.

Correct answer: D. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q86. A notification-fanout agent sending many messages in a short burst starts hitting rate limits on a shared API. What's the reliable way to handle this?

- A. Ignore rate-limit responses and keep sending calls at the same pace.
- B. Abandon the task entirely the first time a rate limit is hit.
- C. ✓ Respect the rate limit proactively — throttle or queue calls and use exponential backoff on 429-style responses, rather than retrying immediately and repeatedly.
- D. Retry every rate-limited call immediately in a tight loop until it succeeds.

Correct answer: C. Proactive throttling and exponential backoff keep the system working within the provider's limits instead of either hammering a rate-limited endpoint or giving up entirely at the first sign of throttling.

Q87. A customer-facing support agent with a committed response-time sla needs to meet a defined reliability bar. What should the design account for?

- A. Treat any SLA miss identically to a total system failure with no graceful handling.
- B. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- C. Assume the agent will always respond in time with no explicit timeout or fallback design.
- D. Set no monitoring and only find out about SLA misses from user complaints.

Correct answer: B. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q88. An internal agent whose output feeds a time-sensitive business process needs to meet a defined reliability bar. What should the design account for?

- A. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- B. Assume the agent will always respond in time with no explicit timeout or fallback design.
- C. Set no monitoring and only find out about SLA misses from user complaints.
- D. Treat any SLA miss identically to a total system failure with no graceful handling.

Correct answer: A. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q89. A production monitoring agent that must reliably detect incidents needs to meet a defined reliability bar. What should the design account for?

- A. Assume the agent will always respond in time with no explicit timeout or fallback design.
- B. Set no monitoring and only find out about SLA misses from user complaints.
- C. Treat any SLA miss identically to a total system failure with no graceful handling.
- D. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.

Correct answer: D. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q90. A compliance-checking agent with a required turnaround time needs to meet a defined reliability bar.

What should the design account for?

- A. Set no monitoring and only find out about SLA misses from user complaints.
- B. Treat any SLA miss identically to a total system failure with no graceful handling.
- C. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- D. Assume the agent will always respond in time with no explicit timeout or fallback design.

Correct answer: C. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q91. A real-time fraud-screening agent gating live transactions needs to meet a defined reliability bar. What should the design account for?

- A. Treat any SLA miss identically to a total system failure with no graceful handling.
- B. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- C. Assume the agent will always respond in time with no explicit timeout or fallback design.
- D. Set no monitoring and only find out about SLA misses from user complaints.

Correct answer: B. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q92. An agent embedded in a checkout flow with strict latency expectations needs to meet a defined reliability bar. What should the design account for?

- A. ✓ Design explicit timeouts, fallback behavior, and monitoring around the SLA — treating a miss as a defined, handled event rather than an open-ended hang.
- B. Assume the agent will always respond in time with no explicit timeout or fallback design.
- C. Set no monitoring and only find out about SLA misses from user complaints.
- D. Treat any SLA miss identically to a total system failure with no graceful handling.

Correct answer: A. Meeting an SLA requires designing for the failure case explicitly — timeouts, fallback paths, and monitoring — rather than assuming the happy path will always hold.

Q93. A long-running multi-step migration task is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Restart the entire task from the very beginning after any interruption.
- B. Keep all state only in memory with no persistence at all.
- C. Abandon the task entirely if it's interrupted partway through.
- D. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.

Correct answer: D. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q94. A multi-hour data-processing job is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Keep all state only in memory with no persistence at all.
- B. Abandon the task entirely if it's interrupted partway through.
- C. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- D. Restart the entire task from the very beginning after any interruption.

Correct answer: C. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q95. An extended multi-turn customer-support session is interrupted partway through by a crash or restart.

What design choice makes recovery reliable?

- A. Abandon the task entirely if it's interrupted partway through.
- B. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- C. Restart the entire task from the very beginning after any interruption.
- D. Keep all state only in memory with no persistence at all.

Correct answer: B. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q96. A long research task with many completed subtasks is interrupted partway through by a crash or restart.

What design choice makes recovery reliable?

- A. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- B. Restart the entire task from the very beginning after any interruption.
- C. Keep all state only in memory with no persistence at all.
- D. Abandon the task entirely if it's interrupted partway through.

Correct answer: A. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q97. A multi-stage document-review pipeline is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Restart the entire task from the very beginning after any interruption.
- B. Keep all state only in memory with no persistence at all.
- C. Abandon the task entirely if it's interrupted partway through.
- D. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.

Correct answer: D. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q98. A long-running batch-classification job is interrupted partway through by a crash or restart. What design choice makes recovery reliable?

- A. Keep all state only in memory with no persistence at all.
- B. Abandon the task entirely if it's interrupted partway through.
- C. ✓ Persist progress and state incrementally so the task can resume from the last completed step rather than restarting from scratch after an interruption.
- D. Restart the entire task from the very beginning after any interruption.

Correct answer: C. Incremental, persisted progress lets a long-running task resume close to where it left off instead of wasting completed work and starting over from zero after every interruption.

Q99. A production agent system currently has a raw stack trace being shown directly to an end user. What's the better design for error propagation to the user?

- A. Suppress all error indication so the user believes everything succeeded.
- B. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- C. Show the raw internal error directly to the user unfiltered.
- D. Always show a generic message with no distinction between different failure types.

Correct answer: B. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q100. A production agent system currently has a generic "something went wrong" message with no actionable detail for the user. What's the better design for error propagation to the user?

- A. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- B. Show the raw internal error directly to the user unfiltered.
- C. Always show a generic message with no distinction between different failure types.
- D. Suppress all error indication so the user believes everything succeeded.

Correct answer: A. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q101. A production agent system currently has an internal error code surfaced with no plain-language explanation. What's the better design for error propagation to the user?

- A. Show the raw internal error directly to the user unfiltered.
- B. Always show a generic message with no distinction between different failure types.
- C. Suppress all error indication so the user believes everything succeeded.
- D. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.

Correct answer: D. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q102. A production agent system currently has a silent failure where the user sees a successful-looking response that's actually incomplete. What's the better design for error propagation to the user?

- A. Always show a generic message with no distinction between different failure types.
- B. Suppress all error indication so the user believes everything succeeded.
- C. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- D. Show the raw internal error directly to the user unfiltered.

Correct answer: C. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q103. A production agent system currently has an error message that reveals internal system details irrelevant or confusing to the user. What's the better design for error propagation to the user?

- A. Suppress all error indication so the user believes everything succeeded.
- B. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- C. Show the raw internal error directly to the user unfiltered.
- D. Always show a generic message with no distinction between different failure types.

Correct answer: B. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q104. A production agent system currently has a timeout that's presented to the user identically to a definitive failure. What's the better design for error propagation to the user?

- A. ✓ Translate the internal error into a clear, accurate, actionable message for the user, without either exposing internals or hiding that something went wrong.
- B. Show the raw internal error directly to the user unfiltered.
- C. Always show a generic message with no distinction between different failure types.
- D. Suppress all error indication so the user believes everything succeeded.

Correct answer: A. Good error propagation is honest and actionable without leaking internal detail: the user should know something went wrong and roughly what to do next, without seeing raw internals or being falsely told everything worked.

Q105. A production system relies on a policy document referenced identically across thousands of support interactions across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Re-send a freshly retrieved copy of the context on every call with no caching.
- B. Let each call use whatever version of the context happens to be locally cached by chance.
- C. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- D. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.

Correct answer: D. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q106. A production system relies on a large product catalog referenced repeatedly across many sessions across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Let each call use whatever version of the context happens to be locally cached by chance.
- B. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- C. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- D. Re-send a freshly retrieved copy of the context on every call with no caching.

Correct answer: C. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q107. A production system relies on a legal-terms reference used consistently across many contract reviews across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- B. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- C. Re-send a freshly retrieved copy of the context on every call with no caching.
- D. Let each call use whatever version of the context happens to be locally cached by chance.

Correct answer: B. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q108. A production system relies on a knowledge base referenced the same way across many customer queries across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- B. Re-send a freshly retrieved copy of the context on every call with no caching.
- C. Let each call use whatever version of the context happens to be locally cached by chance.
- D. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.

Correct answer: A. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q109. A production system relies on a system-wide style guide referenced across many content-generation calls across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Re-send a freshly retrieved copy of the context on every call with no caching.
- B. Let each call use whatever version of the context happens to be locally cached by chance.
- C. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- D. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.

Correct answer: D. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q110. A production system relies on a compliance checklist referenced identically across many audit tasks across many calls. Besides cost, what reliability benefit does caching this stable content provide?

- A. Let each call use whatever version of the context happens to be locally cached by chance.
- B. Periodically regenerate the context from scratch to "keep it fresh," even when nothing changed.
- C. ✓ Cache the stable, repeated context so every call references the identical, consistent version rather than risking subtly different copies drifting across calls.
- D. Re-send a freshly retrieved copy of the context on every call with no caching.

Correct answer: C. Beyond cost savings, caching stable reference content ensures every call is grounded in the same, consistent version — reducing the risk of subtle drift or inconsistency across calls.

Q111. Before a multi-agent research pipeline goes into production, how should its reliability under failure conditions be validated?

- A. Test only the successful path and skip testing any failure scenarios.
- B. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- C. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- D. Only discover failure behavior when it happens for real in production.

Correct answer: B. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q112. Before a customer-support orchestration system goes into production, how should its reliability under failure conditions be validated?

- A. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- B. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- C. Only discover failure behavior when it happens for real in production.
- D. Test only the successful path and skip testing any failure scenarios.

Correct answer: A. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q113. Before a document-processing pipeline goes into production, how should its reliability under failure conditions be validated?

- A. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- B. Only discover failure behavior when it happens for real in production.
- C. Test only the successful path and skip testing any failure scenarios.
- D. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.

Correct answer: D. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q114. Before a payment-approval workflow goes into production, how should its reliability under failure conditions be validated?

- A. Only discover failure behavior when it happens for real in production.
- B. Test only the successful path and skip testing any failure scenarios.
- C. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- D. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.

Correct answer: C. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q115. Before a data-migration agent system goes into production, how should its reliability under failure conditions be validated?

- A. Test only the successful path and skip testing any failure scenarios.
- B. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- C. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- D. Only discover failure behavior when it happens for real in production.

Correct answer: B. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.

Q116. Before an incident-response automation system goes into production, how should its reliability under failure conditions be validated?

- A. ✓ Deliberately test it against injected failures — timeouts, malformed tool results, unavailable dependencies — before relying on it in production.
- B. Assume it's reliable because it worked correctly in a handful of happy-path manual tests.
- C. Only discover failure behavior when it happens for real in production.
- D. Test only the successful path and skip testing any failure scenarios.

Correct answer: A. Deliberately injecting realistic failure modes (timeouts, bad data, unavailable dependencies) during testing surfaces reliability gaps before they show up for real users — happy-path testing alone doesn't.